

Two Sources of Hardness in Generalized Heroes of Might and Magic III Combat

Ivan Parfenchuk
Independent researcher
ursacorp.llc@gmail.com

Abstract

We study the complexity of planning a single round of combat in a generalized version of *Heroes of Might and Magic III* (HoMM3), using the open-source VCMi reimplementation [VCMi26] as the specification of the rules. We show that one-round combat planning has **two independent sources of strong NP-completeness**: pre-battle allocation of an army to slots is strongly hard once the roster carries heterogeneous damage values, and in-battle target selection remains strongly hard even with a fixed, single-type army. Between them we place a positive result—a pseudo-polynomial $O(kB)$ dynamic program—which points at what makes the tractable case tractable, and it is not the poverty of the roster but the triviality of the *reach structure* relating slots to enemies. All four hardness statements—three of them strong, one weak and provably tight—hold under severe restrictions of the game: one round, flat damage, no spells, no heroes, no abilities, and in three of the four no obstacles at all.

Every rule of the model is cited to a line of the VCMi source; the combat arithmetic and the health mechanics are additionally cross-checked against the shipped engine classes themselves. Each reduction was exhaustively regression-tested on bounded instances against an executable transcription of the rules. That process caught five substantive errors, including two in earlier versions of the proofs presented here, and we report them rather than silently correcting them. Finally, on a separate corpus of 145 instances small enough to solve exactly, we measure the gap between the certified optimum and the value of allocations proposed by three tiers of one contemporary language-model family (each allocation played out optimally thereafter), and find a consistent shortfall at the weakest tier that has already vanished at the middle tier.

MSC 2020. 68Q17, 91A46.

ACM CCS. Theory of computation → Problems, reductions and completeness; Applied computing → Computer games.

Keywords. computational complexity; NP-hardness; game AI; Heroes of Might and Magic.

1 Introduction

HoMM3 (1999) is a turn-based strategy game whose battles are fought on a hex grid between armies of at most seven *stacks*, each stack being a number of identical creatures. Before a battle the player distributes creatures among the stacks; during a battle each stack moves and attacks once per round. Both decisions are the ordinary business of playing the game, and both are performed by hand, by every player, before and during every fight.

This paper asks how hard they are.

1.1 The headline

One-round combat planning in generalized HoMM3 has two independent sources of strong NP-completeness: pre-battle allocation is strongly hard with heterogeneous damage types, while in-battle target selection remains strongly hard even with a fixed, single-type army.

The two are independent in a precise sense. The first survives when the board wires each enemy to exactly three deployment cells and the player merely fills the seats; the second survives when the board wires nothing at all, and also when the board wires a great deal but the army is a single creature type deployed one creature per slot. Neither theorem is a restriction or special case of the other: each retains its hardness after the decision on which the other construction rests has been fixed. A natural-looking conjecture that would have unified them—that hardness scales with the diversity of the roster—is false; we state it, and refute it, in Section 3.3.

Between the two we prove a positive result. In the family produced by our first reduction, the problem is solvable by an $O(kB)$ dynamic program, so on that family the weak hardness we prove is exactly the hardness there is. Comparing that family to the hard ones isolates the responsible parameter: the **reach hypergraph**, whose vertices are enemies and whose hyperedges record which slots can engage which enemy. When it is a perfect matching—and every enemy stack is a single creature, so that a blow either finishes its target or scores nothing—the allocation separates into independent per-slot threshold decisions and the problem is a knapsack (Proposition 1.1 states the exact hypotheses; with multi-creature stacks partial kills score and the threshold framing fails). Every hard family we construct breaks the matching—a slot engaging several enemies, or several slots the same enemy—and strong hardness then appears with a rich roster, with a poor one, and with no allocation decision at all.

We do not claim a dichotomy. We have hard cases on both sides of the roster axis and a tractable case with a degenerate reach structure; that is a description of the examples we have, not a theorem about all restrictions—trivially easy non-matching reach structures exist (an instance consisting of a single enemy reached by two slots, say), so “as soon as the matching breaks, hardness appears” would be false and we do not claim it.

1.2 Contributions

1. **A formal model of HoMM3 combat cited line by line to the VCM engine** (Section 2), with an executable transcription, and with the combat arithmetic and health mechanics cross-checked against the shipped engine classes.
2. **Theorem 1** (Section 3.1): allocation is NP-complete with a *single* creature type, $R = 1$, no obstacles, one creature per enemy stack, and a one-row battlefield. Weak hardness, from PARTITION [GJ79, SP12].
3. **Proposition 1.1** (Section 3.1): on the family the reduction *constructs*—which additionally has persistent matching reach; it and Theorem 1’s one-creature-per-enemy-stack restriction are both load-bearing—an $O(kB)$ dynamic program solves the problem, so Theorem 1 is tight and strong hardness there is impossible unless $P = NP$.
4. **Theorem 2** (Section 3.2): allocation is *strongly* NP-hard, from 3-PARTITION—NP-complete in the strong sense [GJ79, SP15]—with every *player* creature type of stock one and one creature per enemy stack.

5. **Theorem 3 and Corollary 3.1** (Section 3.3): the problem is strongly NP-hard with a **single creature type**, and remains so with the allocation fixed to one creature per slot. The reduction is from planar exact cover by 3-sets and uses the engine’s lower clamp on damage as its source of arithmetic. This refutes the roster-diversity conjecture.
6. **Theorem 4 with Corollaries 4.1 and 4.2** (Section 3.4): strong NP-hardness survives three further restrictions—an obstacle-free rectangle with *complete reachability* (in the starting position, every stack can attack every enemy); a given allocation, which isolates unrestricted target assignment as the hard decision; and an objective changed from creatures killed to hit points removed.
7. **A machine-checking methodology** (Section 4) and an honest report of the errors it caught. One scope note belongs up front: the checks exercise the constructions on bounded instances; the embedding algorithm of Lemma D.4 is implemented step by step—with three named subroutine substitutions, disclosed in Appendix D.6—and validated on the whole instance corpus (Section 4.4), but its correctness on *all* inputs—like every proof in this paper—rests on the hand proof, not on a proof assistant.
8. **An empirical study** (Section 5) comparing exactly computed optima against allocations proposed by three tiers of language models on 145 instances, each allocation completed by oracle-optimal play.

1.3 What is not new

The combinatorial cores are standard. Theorem 2 and Theorem 4 use the 3-PARTITION targeting structure already seen in Hearthstone puzzles [HLW20]; Theorem 3 is planar exact-cover targeting; the homogeneous-resource split of Theorem 1 is the mechanism of [FB10, Thm. 7], published in 2010. We claim no new reduction technique. We claim three things: a *separation*—contrasting restricted families that locate where HoMM3’s hardness resides; the fact that the decisions in question are literal player-facing actions of a shipped game rather than modelling devices; and a standard of verification uncommon in this genre. Section 6 sets out the neighbours in detail.

2 The model

2.1 Why the game must be generalized

Shipped HoMM3 has a fixed battlefield of 11×17 hexes, at most 7 army slots per side (rule R14, Appendix B), and a fixed creature roster. A game with a bounded state space is decidable in constant time, so we follow the standard practice of the genre and generalize: the battlefield is $n \times m$, the number of slots is k , and creature statistics are arbitrary integers given in the input. Numbers are written in binary unless stated otherwise; binary is the honest encoding for a game that routinely carries stack sizes in the tens of thousands.

One further thing is generalized, and it deserves its own flag. **The deployment cells are part of the input.** In the shipped game a stack’s starting hex is a function of the slot count and slot index, read from fixed formation tables (R14); in our problem the scenario prescribes them, as it prescribes obstacles and enemy placement. The object we study is therefore *generalized battle-scenario planning with prescribed deployment cells*, and a claim that “only the bounds are generalized” would be false. All four theorems use this

freedom; Theorem 3 uses it heavily. Recovering any of them under native formations is open (Section 7).

A second, smaller generalization was found by external review, and we record it rather than let it be found again. The engine does **not** make every cell of its rectangle usable: the first and last columns fail `BattleHex::isAvailable()` and are labelled `SIDE_COLUMN` by `getAccessibility` (R16). Our constructions do use column 0—Theorem 1 puts p_1 there, Theorem 2 deploys its first group in it ($q_1^1 = (0, 1)$ is a deployment hex, which under R16 could not be occupied at all), and Theorem 4 puts p_1 in it (its line $p_j = (j - 1, 0)$ runs along the top row, so only p_1 touches column 0). So “the battlefield is $n \times m$ ” is generalized in one further respect: we treat every cell as usable. The repair is mechanical: pad with one unusable column on each side and shift every constructed coordinate one **column** to the right—the row must not change, since shifting y flips row parity and is not adjacency-preserving on the offset grid. **Speeds must be left alone—for Proposition 1.1.** An earlier version of this paragraph also prescribed widening the speed bounds by two. Translation preserves every hex distance, so both directions of Theorem 1 carry over unchanged (its no-direction is in fact reach-independent; Appendix E); but raising the player’s speed from 2 to 4 raises its strike radius from 3 to 5, and on the PARTITION instance (2, 4) the second slot then reaches E_1 as well as E_2 , so the family no longer has the persistent matching reach that Proposition 1.1 assumes. (We checked this against the executable model rather than by inspection.) For Theorem 3, horizontal translation preserves row parity, adjacency, distances and components, so (I1)–(I3) survive and (I4) does too since no region grows; if σ is defined as the board’s hex count it grows by twice the height, not by two. The shift has not been carried out in the constructions or the verifiers as they stand, so we flag it here as a gap between the model and the engine rather than claim it away.

2.2 The rules, with citations

We use VCMi [VCMi26], an open-source reimplementaion whose combat module reproduces the original game’s damage numbers, as the specification. Every rule below carries a tag R1–R17; Appendix B maps each tag to the exact `file:line` of the checkout (commit `b5cee70`), so the body can be read without recourse to the paths. The full model is `MODEL.md`; this is the fragment the theorems need.

Battlefield. A hex grid in offset (“even-row shifted”) coordinates with six neighbour directions (R1); distance is computed through axial coordinates (R2). A subset of hexes may be impassable, merging the engine’s three mechanisms—battlefield `impassableHexes`, obstacle objects, and siege walls (R3). **A hex occupied by a living unit is not enterable, and a dead unit stops blocking** (R4), which matters a great deal in Section 3.3.

Units. A creature type is $(att, def, dmg_{\min}, dmg_{\max}, hp, spd, flags)$, plus a nonnegative *value* used by the objective. A stack is a type together with a creature count, and its health is stored as a pair $(fullUnits, firstHPleft)$ representing a pool $avail = firstHPleft + hp \cdot fullUnits$ (R5); a fresh stack of c creatures has $fullUnits = c - 1$ and $firstHPleft = hp$, so $avail = c \cdot hp$ and $count = c$. The rule the reductions live on is the **effective count** (R6):

$$count(S) = fullUnits + [firstHPleft > 0].$$

A stack’s offensive output is proportional to count, not to *avail*. A stack of one Archangel reduced to 1 hit point out of 250 deals exactly as much damage as a healthy one. Damage output is a **step function** of damage taken.

Damage (R7). With $\Delta = att(attack) - def(defender)$:

$$\begin{aligned} f_{att} &= 1 + \min(0.05\Delta, 4.0) \quad \text{if } \Delta > 0, \text{ else } 1 \\ f_{def} &= 1 - \min(0.025(-\Delta), 0.7) \quad \text{if } \Delta < 0, \text{ else } 1 \\ dmg &= \max(1, \lfloor count \cdot d \cdot f_{att} \cdot f_{def} \rfloor) \end{aligned}$$

In the mathematical model the constants are the exact rationals $1/20$, $1/40$, 4 , $7/10$ and the floor is exact integer arithmetic; the engine evaluates the same formula in IEEE-754 doubles, and Section 4.2 documents the one divergence the cross-check has found—a claim scoped to the cases run, not a proof that no other exists. Two features of this formula do real work below. The multiplicative dependence on count makes damage linear in stack size; the **lower clamp at 1** breaks that linearity from below, so that splitting a stack into singletons can deliver strictly more total damage than keeping it whole. Section 3.3 is built entirely on the clamp.

Kills and overkill (R8). Damage accumulates in the defender’s pool; excess beyond the pool is discarded (the *overkill rule*), and

$$kills(D) = \begin{cases} 0 & \text{if } D < firstHPleft \\ \min\left(1 + \left\lfloor \frac{D - firstHPleft}{hp} \right\rfloor, count\right) & \text{otherwise.} \end{cases}$$

Turn structure. Play proceeds in rounds; within a round each living stack acts once in decreasing order of *spd*, ties broken by side then slot index (R9). That tie rule is a **simplification**: the engine alternates sides on equal initiative according to which side moved last, and gives the attacker priority only on the first turn (R17). We use the simplified rule because in every construction here each player stack is strictly faster than every enemy, so no equal-initiative tie between the sides ever arises and the two rules coincide on the instances we build; but the simplification is real and the executable transcription shares it, so the mechanics tests cannot detect the difference. What the tests cannot detect, the engine harness pins directly: two dedicated tie cases run the engine’s own queue and confirm the alternation rule is exactly as stated here (Section 4.1). **Speed is simultaneously initiative and movement range** (R10)—a construction cannot set them independently, which rules out the obvious “slow but far-reaching” gadget. Stacks that use **WAIT** act after all **NORMAL** stacks, in *increasing* speed order (R9)—the fact the garrison policy of Section 2.4 rests on, and one the engine harness executes through the engine’s own queue (Section 4.1). Movement is unweighted BFS over enterable hexes bounded by *spd*; **WALK_AND_ATTACK** moves to a hex adjacent to the target and strikes (R11), so a melee stack of speed s can strike only enemies at distance at most $s + 1$; whether it can actually strike such an enemy depends on the free hexes available.

Retaliation (R12). The attacker’s blow resolves first; the defender retaliates afterwards if it is still alive, has a charge left, and the attacker lacks **BLOCKS_RETALIATION**. **A stack killed outright does not retaliate.** Charges reset at the round boundary, so the first attacker into a stack absorbs the retaliation and later attackers in the same round strike free.

Determinization. We restrict to instances with $dmg_{\min} = dmg_{\max}$, exclude morale, luck, spells, heroes, and every conditional damage multiplier, and call the result **H3-det** (MODEL.md §7). Constraining the *instances* rather than reinterpreting the *rules* is deliberate: **H3-det** is then a genuine special case of the generalized model of Section 2.1—not of the shipped game, whose board, roster and formations are fixed—so hardness transfers upward to that model, and no rule was reinterpreted to make a proof work. One further restriction deserves its own name. Every construction in this paper uses only melee, single-hex creatures with the default single retaliation charge—the fragment **H3-det-melee** (MODEL.md Definition 7.1a), on which the damage formula above is the *whole* formula: no ranged or distance penalties, no breath or multi-target attacks, no double-wide movement cases. Our hardness results are therefore statements about **H3-det-melee**, and they transfer upward to **H3-det** and to the generalized model because a restriction of the instance family only strengthens a hardness claim; the empirical instances of Section 5 live in the same fragment.

2.3 The problem

Problem (ARMY-ALLOCATION). **Input.** A battlefield $(n, m, obstacles)$; k slots with deployment hexes $p_1, \dots, p_k$; a multiset A of player creatures as (type, count) pairs; a fixed defence—enemy stacks with types, counts and hexes—which plays the fixed policy ($\dagger$) of Section 2.4; a round bound R in unary; a target $W \in \mathbb{Z}_{>0}$. Every creature type carries a nonnegative integer *value* as part of its tuple (Section 2.2); the objective reads the values of enemy types only. The deployment hexes $p_1, \dots, p_k$ are pairwise distinct, passable, and distinct from every enemy hex, and an allocated stack begins the battle on the deployment hex of its slot. Where R9’s speed comparison ties across the two sides, the player’s stack acts first.

Question. Is there an allocation of A to the k slots (each slot receiving at most one type, each type’s total at most its stock) and a sequence of player actions such that after R rounds against ($\dagger$) the total value of enemy creatures killed is at least W ?

BATTLE-PLAY is the same question with the allocation given as part of the input, so ARMY-ALLOCATION contains BATTLE-PLAY as a special case.

Three points on the formulation. The enemy plays one concrete scripted policy, not an adversarial one, which keeps the problem in NP and matches the informal “against a fixed defence”; the adversarial version is a different problem. Baking ($\dagger$) into the problem rather than taking an arbitrary “deterministic poly-time policy π ” as input is deliberate: an arbitrary encoded program with a promised running time is not a syntactically checkable input restriction, a circuit encoding would repair that at no gain, and every theorem uses ($\dagger$) anyway—hardness for the one fixed policy is the stronger statement. The objective counts **whole creatures killed**, weighted by the per-type value. That is the game’s own accounting: a stack at one hit point fights at full strength, so hit points removed without a kill buy the player nothing in that round. It is *not*, however, the sole source of hardness—counting hit points removed collapses only the matching-reach family of Theorem 1 to a separable concave sum; on the family of Theorem 4 the hit-point objective remains strongly NP-hard (Corollary 4.2). And R is given in unary so that evaluating a certificate is polynomial.

Lemma 2.1 (membership). $\text{ARMY-ALLOCATION} \in \text{NP}$.

Proof. Fix the encoding: the battlefield is listed hex by hex and R is unary. A certificate is the allocation together with the player’s actions: for each stack-round pair, an optional

WAIT bit and one terminal action: DEFEND, a destination hex (move), or a (destination hex, target) pair (WALK_AND_ATTACK)—at most $2kR$ action tokens, since a stack that waits still takes its terminal action later in the same round (Section 2.2). Simulating the battle takes a polynomial number of arithmetic operations and BFS computations, (§) is computable in constant time, and comparing destroyed value to W completes the check. ■

2.4 The garrison policy

All four theorems use the same scripted defence, and it has to be pinned down precisely, because “hold position” does not determine an action: H3-det retains movement, WAIT, and DEFEND.

(§) If the stack has not waited this round, issue WAIT; on its postponed activation, issue DEFEND at its current hex.

Both are shipped actions. WAIT postpones the stack’s terminal action into the round’s WAIT phase, which runs after all NORMAL-phase activations, in *increasing* speed order (R9); DEFEND (R13) ends the turn without moving or attacking and grants +20% defence—an integer bonus with a floor of +1—until the stack next receives a turn. Three consequences are used throughout:

- no enemy ever initiates an attack, so no player stack ever delivers retaliation damage;
- a waiting or defending enemy still retaliates when struck—neither action consumes the retaliation charge;
- **the one-round lemma:** in round 1, the blow of a player stack that did not wait lands at its NORMAL-phase activation, before any enemy’s postponed DEFEND, and therefore meets the un-raised defence—*regardless of relative speeds*; and every blow, waiting or not, delivers at most its nominal damage, since raising the target’s defence can only lower f_{def} . **Every statement quantifying over an arbitrary play is therefore phrased as “at most the nominal damage”;** equalities are asserted only for the constructed witness plays, which never wait.

The lemma’s scope is exactly one round, and both ways it fails outside that scope are worth stating, because an earlier justification (“no blow ever meets the bonus”) claimed too much and was refuted by external review. A player stack that itself waits is scheduled in the WAIT phase by *increasing* speed, so a slower enemy’s postponed DEFEND can land first and the player’s postponed blow then meets the bonus—for such blows only the inequality survives. And for $R \geq 2$ the bonus persists past the round boundary until the enemy’s next activation, so a fast player striking early in round 2 meets the bonus left over from round 1. All four theorems set $R = 1$; a multi-round extension must redo this analysis.

An earlier version of (§) issued DEFEND at the stack’s own turn. For the theorems the two are interchangeable—every constructed player stack is strictly faster than every enemy—but they part company on the empirical corpus of Section 5, whose natural instances contain enemies *faster* than the player, and there the old policy made six recorded optima unattainable (Section 5.1 gives the history and the machine checks that close it). The present (§) was adopted after being verified reduction by reduction by external review; the mechanics it stands on—the WAIT-phase order, the bonus arithmetic and duration, and the retaliation-charge neutrality of both actions—are unit-checked against the cited engine lines, and the searches of Section 4 include a variant in which the defence executes (§) literally, phase by phase (Section 4.4).

That the policy is load-bearing is not obvious. The first version of Theorem 1 used an attacking garrison and was wrong: an enemy that attacks provokes a *retaliation*, which delivers a second blow to that same enemy inside the same round, and the reduction then decides SUBSET-SUM with the wrong budget. Section 4.3 tells the story.

3 Results

We present the theorems in pedagogical order. The two-source claim of Section 1.1 is assembled from Theorems 2, 3 and 4 at the end of Section 3.4.

Throughout, $(\star)$ denotes the specialization of the damage formula used by Theorems 1, 2 and 4: every player type and every enemy type has $att = def = \alpha$ with $\alpha := 1$, and every enemy type has flat damage 1. Then $\Delta = 0$ in both directions, $f_{att} = f_{def} = 1$, and a stack of c creatures of flat per-creature damage d delivers nominal damage cd . Retaliation costs the player nothing under $(\star)$: the defence never initiates (Section 2.4), so a player stack takes at most one retaliation, and it takes it *after* its own blow (R12), so no blow already delivered is affected. Theorem 3 deliberately breaks $(\star)$ and uses the defence factor.

3.1 Theorem 1, and an algorithm that matches it

Theorem 1. ARMY-ALLOCATION is NP-complete, already for instances with $R = 1$, a *single creature type* in the player’s army, one creature per enemy stack, no obstacles, and a battlefield of one row.

The interesting word is *single*. There is nothing to choose about what to bring; the only free variable is how many creatures go in each slot.

Construction. From a PARTITION instance [GJ79, SP12] $a_1, \dots, a_n$ with $\sum a_i = 2B$: one row of $5n$ hexes; block j occupies hexes $5(j-1), \dots, 5j-1$, with the deployment hex $p_j := 5(j-1)$ and the enemy E_j at $e_j := 5(j-1) + 1$. Under $(\star)$ the player has one type with $att = def = 1$, flat damage 1, $hp = 5$, speed 2, no flags, value 0 and stock exactly B ; E_j is one creature of a type with $att = def = 1$, flat damage 1, $hp = value = a_j$, speed 1 and no flags. Set $R = 1$, $W = B$.

Correctness. A stack of speed s can strike only enemies at hex distance at most $s + 1$, in any position and regardless of which hexes are free (Lemma E.3); consecutive deployment hexes are 5 apart, so every foreign enemy sits at distance at least 4 from p_j against a strike radius of $spd + 1 = 3$, and slot j can strike E_j and nothing else—in every position of the play, not only at the start, because the obstruction is distance rather than blocking, and deaths do not change distances. A stack strikes at most once, so the striker sets of distinct enemies are disjoint, and the total damage E_j absorbs is at most c_j (Section 2.4); E_j dies only if $c_j \geq a_j$, and a non-waiting blow with $c_j \geq a_j$ kills. If PARTITION has a solution S , allocate $c_j = a_j$ for $j \in S$: total B , destroyed value B . Conversely, if the game is a yes-instance with dead set S , then $B \leq \sum_{j \in S} a_j \leq \sum_{j \in S} c_j \leq \sum_j c_j \leq B$, so every inequality is tight and S solves PARTITION. On a malformed encoding, a non-positive a_i or an odd $\sum a_i$ the reduction outputs the fixed no-instance of Lemma D.4; on the empty instance $n = 0$ —a PARTITION yes-instance—it outputs $G((1, 1))$, which keeps the map total. With Lemma 2.1 this gives NP-completeness. The full proof is Appendix E. ■

The numbers a_j are carried as hit points in binary, so this is only *weak* hardness—and that is exactly right, because the family admits an algorithm that matches that bound:

Proposition 1.1. On the family of Theorem 1—single creature type of flat damage $d \geq 1$, $R = 1$, policy $(\ddagger)$, damage under $(\star)$, *one creature per enemy stack*, and persistent

matching reach: a bijection $j \mapsto E_j$ such that, whatever the feasible allocation, in every position of round 1 reachable by legal play from its starting position, for every slot j whose stack has not yet taken its terminal action, the set of enemies that stack can strike is exactly $\{E_j\}$ —ARMY-ALLOCATION is solvable in $O(kB)$ time and $O(B)$ space, B being the stock.

Proof. After deployment the play is forced: slot j either finishes E_j or achieves nothing. This is where **one creature per enemy stack** is load-bearing: against a stack of several creatures a non-finishing blow still kills whole creatures and still scores, the per-slot value is a staircase in c_j rather than a threshold, and the 0-1 framing below is false (open problem 4, Section 7—a later review pass exhibited a matching-reach instance with six-creature stacks where the threshold rule returns 0 and the true optimum is 3). Against a single creature of t_j hit points, slot j scores v_j iff its nominal damage $c_j d$ —delivered exactly by a non-waiting blow under $(\star)$, since $\Delta = 0$ makes both factors 1; a waiting blow meets the postponed DEFEND bonus and delivers at most nominal, which only helps the upper-bound direction—reaches t_j , i.e. iff $c_j \geq b_j := \lceil t_j/d \rceil$ (well-defined since $d \geq 1$), and surplus is wasted. The optimum is $\max\{\sum_{j \in S} v_j : \sum_{j \in S} b_j \leq B\}$, a 0-1 knapsack over k items, solved by the textbook dynamic program. The full proof, with the hypothesis consumed exactly where it is needed, is Appendix E. ■

Proposition 1.1 is machine-checked twice: the knapsack program against exhaustive subset enumeration of the same abstraction on 2000 random instances, and—since round 10—against exhaustive play of built corridor instances in the game model itself, including a negative control with multi-creature enemy stacks on which the suite asserts that the threshold rule and the game *disagree* (`dp_single_type.py`). It matters for three reasons. It shows Theorem 1 is tight. It disposes of an apparent contradiction—“Theorem 1 gives a DP, Theorem 2 forbids one”—which confuses two incomparable restrictions of the same general problem. And, read against Theorems 3 and 4, it identifies the parameters that are actually responsible: not the roster, but the matching reach structure together with single-creature stacks (open problem 4 asks what survives without the latter).

3.2 Theorem 2: allocation-driven strong hardness

Theorem 2. ARMY-ALLOCATION is **strongly** NP-hard, already for $R = 1$, no obstacles, no abilities, one creature per enemy stack, and instances in which every player creature type has stock exactly one.

Construction. From a 3-PARTITION instance $(a_1, \dots, a_{3m}; T)$ with $\sum a_i = mT$ and $T/4 < a_i < T/2$ —NP-complete in the strong sense [GJ79, SP15]—take three rows and $8m + 2$ columns. For each group g , an enemy E_g with $att = def = 1$, flat damage 1, $hp = T$, speed 1, no flags and value 1 sits at $(X_g, 1)$ with $X_g = 8(g - 1) + 1$, and its three deployment hexes are three of the six neighbours of $(X_g, 1)$, named explicitly: row 1 is odd, and $q_g^1 = (X_g - 1, 1)$, $q_g^2 = (X_g, 0)$, $q_g^3 = (X_g, 2)$, all inside the three-row board. The player has $3m$ types, type C_i with $att = def = 1$, flat damage a_i , $hp = 5$, speed 2, no flags, value 0 and **stock one**. $R = 1$, $W = m$.

Correctness. Each q_g^r is adjacent to E_g and at hex distance at least 7 from every other enemy (tight, attained by q_{g+1}^1 against E_g) against a strike radius of $spd + 1 = 3$, so the three slots of group g strike E_g and nothing else. Each stack holds at most one creature (stock one; slots may stay empty), a stack strikes at most once, and under $(\dagger)$ no enemy initiates, so no player stack ever delivers a retaliation blow (Lemma E.2); hence the damage available to E_g is at most $\sum_{i \in S_g} a_i$ where S_g is the set of types allocated to group

g 's slots, and the S_g are pairwise disjoint. Killing all m enemies forces $\sum_{i \in S_g} a_i \geq T$ for every g ; summing against $\sum a_i = mT$ makes every inequality tight, and $T/4 < a_i < T/2$ forces $|S_g| = 3$. The sufficiency direction, where the game semantics are discharged—all three stacks of a group strike in the NORMAL phase before the enemy's postponed DEFEND, and their damage accumulates in the health pool (R8)—is Lemma E.13; the degenerate encodings are routed as in Theorem 1. The full proof is Appendix E. ■

Remark. The three rows are not decoration. An earlier version of this construction used a single row, as Theorem 1 does. That is impossible: in one row a hex has two neighbours, so a third stack cannot reach E_g without walking through a hex occupied by an ally, and occupied hexes are not enterable. The bug was found by a geometry self-check that verifies the reachability lemma on the *built* instance with all slots occupied. See Section 4.

3.3 Theorem 3: a single type is enough

Theorems 1 and 2 together tempt one into a story: hardness scales with the diversity of the roster, since the single-type family of Theorem 1 has a pseudo-polynomial algorithm and strong hardness needed $3m$ distinct types. An earlier draft of this work recorded that story as an explicit conjecture. It is false.

Theorem 3. ARMY-ALLOCATION is *strongly* NP-hard, already for $R = 1$, a *single creature type* in the player's army, one creature per enemy stack, all enemy creatures of one type and value 1, flat damage, no abilities, and static impassable hexes as the only terrain feature. With Lemma 2.1 it is strongly NP-complete on this family.

Corollary 3.1 (fixed allocation). *The same instances remain strongly NP-hard when the allocation is given: fix one creature in every slot. BATTLE-PLAY is strongly NP-hard on single-type instances.*

Corollary 3.1 is the form to quote, for a reason worth stating plainly. In every yes-instance of Theorem 3 the winning allocation is unique and is the all-ones vector—the budget forces $3q$ creatures into $3q$ slots, one each. There is therefore no interesting sizing decision, and it would be wrong to advertise this as “single-type stack sizing is strongly hard”. What the theorem establishes is that **target selection on the reach hypergraph is strongly hard**, and that it stays hard when the roster is reduced to one type and the allocation is removed from the problem altogether. That is precisely what kills the roster-diversity conjecture.

Source problem. PLANAR-X3C: exact cover by 3-sets whose element/set incidence graph is planar. Dyer and Frieze prove exactly this problem NP-complete—their Lemma 2.2 states “Planar X3C is NP-complete”, with planarity defined on precisely this incidence graph [DF86, p. 175]—and their instances additionally have every element in two or three sets (p. 178), so the hardness survives that restriction too.

The arithmetic. One player type P with $att = 1$, $def = 1$, flat damage 1, $hp = 4$, speed σ equal to the board's hex count, stock exactly $3q$; one enemy type Q with $att = 1$, $def = 27$, flat damage 1, $hp = 3$, speed 1, value 1. Then $\Delta = -26$ and $0.025 \cdot 26 = 0.65$, below the cap 0.7, so $\mu := 0.35$ and a player stack of c creatures delivers nominal damage

$$D(c) = \max(1, \lfloor \mu c \rfloor) : \quad D(1), \dots, D(12) = 1, 1, 1, 1, 1, 2, 2, 2, 3, 3, 3, 4.$$

Lemma 3.2 (resource lemma). *Let $0 < \mu < 1$ and $D(c) = \max(1, \lfloor \mu c \rfloor)$. If stacks of sizes $c_1, \dots, c_r \geq 1$ each deliver at most $D(c_i)$ to one target and the total is at least 3, then $\sum_i c_i \geq 3$, with equality iff $r = 3$ and $c_1 = c_2 = c_3 = 1$.*

Proof. For $c = 1$, $D(1) = 1 = c$; for $c \geq 2$, $\lfloor \mu c \rfloor \leq \mu c < c$ and $1 < c$, so $D(c) < c$. Hence $3 \leq \sum_i D(c_i) \leq \sum_i c_i$. If $\sum_i c_i = 3$ then $\sum_i D(c_i) = \sum_i c_i$, so $D(c_i) = c_i$ and thus $c_i = 1$ for every i ; since $\sum_i c_i = 3$, $r = 3$. ■

This is the damage floor doing the work. A lone creature always delivers a full point no matter how outclassed; c creatures in one stack deliver $\lfloor \mu c \rfloor < c$. Splitting into singletons is strictly better—a real if inelegant HoMM3 tactic—and the granularity it buys is what a covering problem needs. Concretely, three damage costs 9 creatures in one stack, 7 in two, and 3 in three.

Stating the lemma for all $\mu \in (0, 1)$ rather than for 0.35 is deliberate, because the construction’s two branches sit on opposite sides of the defence cap. An undefended blow sits at $0.025 \cdot 26 = 0.65$, clear of the cap. A defending enemy (Section 2.4) has defence 32, hence $0.025 \cdot 31 = 0.775$ —*past* the cap, so that branch is clamped to 0.7 and $\mu = 0.3$; and VCMi’s hand-written JSON parser loads the cap constant 0.7 as 0.7000000000000001 (Section 4), so the clamped branch lives in two slightly different arithmetics, engine and model disagreeing by one unit in the last place (ULP) exactly at integer boundaries of $base \times 0.3$. The lemma is indifferent to all of it: every μ in play—0.35, 0.3, or the engine’s 0.29999999999999993—lies in $(0, 1)$, which is the only property the proof consumes.

The board. Four invariants carry the whole correctness argument, and nothing below refers to the layout:

- **(I1)** every enemy hex z_S has exactly three free neighbours, pairwise non-adjacent;
- **(I2)** the free non-enemy hexes fall into exactly $3q$ connected components, one per element—the **region** R_e ;
- **(I3)** R_e holds the deployment hex p_e and exactly the dockings d_S^e for $S \ni e$, each adjacent to z_S alone;
- **(I4)** the speed σ satisfies $\sigma \geq \max_e |R_e|$, the maximum taken over all elements.

(I1) is not a design choice. The only pairwise non-adjacent triples among a hex’s six neighbours are the two *alternating* triples, so an enemy that must be reachable simultaneously from three mutually sealed regions is forced to present one. This is the single place where hex geometry does real work, and it is why the construction would not transfer unchanged to a square grid.

The local picture is worth having in mind for the whole proof (# impassable, Z the enemy E_S , U/R/D its three dockings—the alternating triple; even rows drawn half a step right, the engine’s convention; the three corridors leave the box toward the three regions R_e , $e \in S$):

```

# . #           to region R_e1
# U #
# Z R . .       to region R_e2
# D #
# . #           to region R_e3
```

While E_S lives, Z is occupied and each region’s corridor is a dead end: the regions are mutually sealed. When E_S dies, Z becomes free and turns into a *doorway* joining the three regions. In the tight plays that decide the no-direction, however, the budget forces every stack to be a singleton, so a dead E_S was killed by three singletons standing on U , R and D —and they never move again, so the doorway opens already plugged. The confinement lemma below is the formal version of this picture.

Correctness, in three steps. Under (I1)–(I4) the stack of slot e can strike exactly the E_S with $S \ni e$, by one approach hex each (**reach**). Since each enemy has 3 hit points and a stack *strikes* at most once per round (acting once would be the wrong invariant under WAIT—Appendix D, Lemma D.6), the striker sets of distinct dead enemies are disjoint; Lemma 3.2 then gives $3t \leq \sum_e c_e \leq 3q$ for t kills, so $t \leq q$, and at $t = q$ every inequality is tight, forcing $3q$ singleton stacks in $3q$ slots, each striking a dead enemy, three per enemy (**budget**). Finally—the least obvious step—a dead enemy stops blocking its hex, so every kill opens a doorway between three regions. The **confinement** lemma says the doorway only ever opens for stacks that have already spent their *terminal* action. The induction runs on the realized order of terminal actions; a stack that issues WAIT merely postpones its terminal action to the later WAIT phase, and the induction reaches it there. A dead E_S was killed by exactly three singleton stacks standing on its three dockings, which by induction are the stacks of the three slots of S ; so if $e \in S$, slot e has already taken its terminal action. The equivalence with PLANAR-X3C follows. The full proof, including the general embedding lemma written out in full, is Appendix D.

The embedding. The remaining content is that the invariants can be realized. Given a planar incidence graph, split each element vertex of degree d into a path of d vertices along the rotation order; this preserves planarity, and the cyclic order of the incident edges becomes the linear order along the path. Then draw the resulting max-degree-3 plane graph orthogonally on a polynomial grid. The drawing theorem needed is Tamassia–Tollis [TT89], quoted here—as in Appendix D—in the form of [DG13, Thm. 7.3]: a **connected** 4-plane graph admits an orthogonal grid drawing in $O(n^2)$ area, with at most four bends per edge, computable in polynomial time (linear, per [TT89]). (Appendix A records exactly what was verified from which source.) The connectivity hypothesis is not free and splitting element vertices does not supply it: the incidence graph of $X = \{1, \dots, 6\}$, $C = \{\{1, 2, 3\}, \{4, 5, 6\}\}$ is a planar yes-instance with two components. Each component is drawn separately; the proof packs the finished *boards* side by side behind impassable strips (Appendix D, step 2), while the implementation packs the *drawings*—two empty grid columns between components, so 2λ hexes after scaling—which the same separation inequality covers (the third named substitution of Appendix D.6). Either way the packing is sound because the invariants (I1)–(I3) are local to a region and (I4) only grows. Then scale by an even factor $\lambda = 20$, replace a 9×9 box around each vertex by a gadget, and declare everything else impassable. The separation inequality is explicit: gadget boxes have radius $\rho = 4$, so features non-incident in the drawing land at L_∞ distance at least $\lambda - 2\rho = 12 \geq 2$ and hex adjacency, which requires L_∞ distance 1, is impossible between them. (An earlier draft used $\lambda = 9$, giving $9 - 8 = 1$; two unrelated boxes could then touch. The repair is this inequality.) Set-vertex boxes need a hand-built adapter, because an orthogonal drawing delivers three edges along three of the four *axis* directions while the three dockings are an alternating triple at 120° ; there are four cases, all four patterns are machine-checked, and all four are printed in Appendix C.

What this changes. The $O(kB)$ dynamic program of Proposition 1.1 is a statement about a matching reach structure, not about a poor roster; Theorem 2 is not the strong endpoint of a diversity axis but a second, independent source of hardness, with many types and a fixed, non-adaptive reach structure—a disjoint union of 3-stars, which leaves the player no targeting choice even though it is not a matching. Both extremes are hard. We drop the conjecture rather than restate it.

The price. Theorem 3 needs obstacles, which Theorems 1, 2 and 4 do not, and it leans hardest on the prescribed deployment cells of Section 2.1, since the whole reduction is carried by which slot sits where.

3.4 Theorem 4: remove the board entirely

Theorem 3 makes the reach structure as rich as planar incidence allows. The opposite extreme is a board that imposes nothing.

Theorem 4. ARMY-ALLOCATION is **strongly** NP-hard already for instances with $R = 1$; **no obstacles and no abilities of any kind**; every player creature type of stock one, one creature per enemy stack; a rectangular open battlefield of six rows and $4m+2$ columns; and **complete reachability**—in the starting position, with every slot occupied, every player stack can attack every enemy stack.

Corollary 4.1. The same instances are hard with the allocation given. BATTLE-PLAY is strongly NP-hard on obstacle-free boards with complete reachability.

Corollary 4.2 (hit-point objective). On the same instances, replace the objective by **total enemy hit points removed**—damage absorbed by the defence, with overkill discarded. Formally, for a play π put

$$\text{absorbed}(\pi) := \sum_{\text{enemy stacks } E} \min(\text{total damage directed at } E, \text{ its initial pool}),$$

and replace the question by “is $\text{absorbed}(\pi) \geq W_{\text{hp}}?$ ” with the target $W_{\text{hp}} = mT$. The problem remains strongly NP-hard, with the allocation free or given.

Proof sketch of Corollary 4.2. Total nominal player damage is $\sum a_i = mT$ and each enemy absorbs at most its pool T , so removing mT hit points forces zero waste: every stack strikes, no blow is reduced (a waiting stack that meets a DEFEND bonus delivers **at most** nominal—not strictly less, since the damage formula clamps at 1 and $a_i = 1$ is legal, and the argument needs only the inequality), and every enemy absorbs exactly T . The striker sets are disjoint and $T/4 < a_i < T/2$, so the tight groups are triples and form a 3-partition; conversely a 3-partition witness play delivers exactly T to each enemy. Machine-checked on the same instance families as the theorem (`scripts/verify_hp_objective.py`): the hit-point relaxation reaches mT exactly on the 3-PARTITION yes-instances, and the witness play’s absorbed total, read off the simulator, is exactly mT on every one. ■

Corollary 4.2 is owed to an external review pass, which caught an earlier draft claiming that a hit-point objective “would make the problem trivial”. That claim is true only where the reach structure is a matching (Theorem 1); in general the kill-counting objective is the natural one, but it is not the sole source of hardness.

Construction. From 3-PARTITION again—NP-complete in the strong sense, as in Theorem 2 [GJ79, SP15]: six rows, $4m + 2$ columns, enemy E_g with $\text{att} = \text{def} = 1$, flat damage

1, $hp = T$, speed 1, no flags and value 1 at $(4g - 2, 3)$; player types $C_1, \dots, C_{3m}$ with $att = def = 1$, flat damage a_i , $hp = 5$, no flags, value 0, stock one, and speed $s = 4m + 8$, deployed along the top row at $p_j = (j - 1, 0)$. $R = 1$, $W = m$.

Correctness. The upper-bound direction uses no geometry at all: a stack takes one terminal action per round (R9, R11), so it strikes at most once and the striker sets of distinct enemies are disjoint, while stock one caps each blow at a_i (a waiting blow delivers at most a_i , which only helps this direction); killing all m enemies forces $\sum_{i \in S_g} a_i \geq T$, which sums to a 3-partition as in Theorem 2. The geometric work is all in the yes-direction, where we choose the play. For E_g at $e_g = (X_g, 3)$, $X_g = 4g - 2$, the three approach hexes are $q_g^1 = (X_g - 1, 2)$, $q_g^2 = (X_g, 2)$, $q_g^3 = (X_g + 1, 3)$ —genuine neighbours of e_g under R1, pairwise distinct across all $3m$ seats (Lemma E.15). Row 1 is never occupied along an attack-only play (Lemma E.16), so a stack still on $p_j = (j - 1, 0)$ steps into row 1, walks along it, and descends to its seat, spending at most $w + 2 = 4m + 4 \leq s = 4m + 8$ movement points regardless of the activation order (Lemma E.17); hence complete reachability holds with all $3m$ slots occupied, and every three-per-enemy assignment φ is simultaneously realizable (Lemmas E.18 and E.19). The degenerate encodings are routed as in Theorem 1. The full proof is Appendix E.

Two honest qualifications. First, the confinement argument that keeps row 1 clear holds along *attack-only* plays—those in which every stack either passes or performs WALK_AND_ATTACK. In the full model a stack may also move without attacking, wait, or defend, and then confinement can fail. This costs nothing: confinement is used only in the yes-direction, where the play is ours to choose, and the no-direction is geometry-free. An earlier draft stated the lemma universally, which was a wording error. Second, the board does impose one thing—an enemy has six neighbours, so at most six stacks can strike it in one round (under the completed $(\star)$ no striker dies to the retaliation it draws, so no seat is vacated mid-round and the capacity is exactly six). That bound never binds here (three per enemy), but “featureless” means *complete reachability plus local seat capacity*, not “positions do not exist”.

Where the hardness lives, and the two-source claim. In Theorem 4 every type has stock one and every slot reaches every enemy, so *every* injection of types into slots is equivalent: the multiset of blows available does not depend on which slot holds which type, and every three-per-enemy targeting is realizable from any of them. The decision that encodes the 3-partition is the choice of targets. Hence Corollary 4.1, and hence Table 1:

	what forces the grouping	what the player decides	hardness
Theorem 1	board adjacency (matching reach)	how many creatures per slot	weak, and tight (Prop. 1.1)
Theorem 2	board adjacency (three seats per enemy)	how to fill the seats	strong
Theorem 3	planar incidence reach, single type	whom to hit	strong, allocation fixable
Theorem 4	nothing	whom to hit	strong, allocation fixable

Table 1: Where the combinatorics sits in each theorem.

Theorem 2 is allocation-driven: the play is forced and the allocation carries the 3-partition. Theorems 3 and 4 are targeting-driven: the allocation is free, or fixed, and the play carries the combinatorics. Neither collapses into the other—Theorem 2 has no targeting choice and a rich roster, Theorem 3 a rich reach structure and a roster of one—and that is the two-source statement of Section 1.1.

4 Machine-checking methodology

Papers in this genre describe their rules in prose and their reductions on paper, and stop there. We did not stop there, and the process is a contribution in its own right—a modest one, but the one that changed the content of this paper most.

Throughout, *review round n* refers to the n -th pass of the external review process this manuscript went through; the numbering is internal and is retained only so that each fix can be traced to the pass that forced it.

4.1 The three layers

1. **An executable transcription of the rules.** `scripts/homm3_model.py` implements Section 2 with the citations in comments. No verifier restates a rule; every check calls the transcription. Unit tests against hand-computed engine numbers: 90 checks, including the WAIT/DEFEND phase mechanics the policy (§) stands on.
2. **Exhaustive regression testing of each reduction on bounded instances.** For small instances we enumerate the allocation space and, for each allocation, search the plays of the **attack-only fragment**, branching over targets **and** over every legal approach hex. What “the allocation space” means differs by suite, and the difference should be stated rather than blurred: Theorem 1’s PARTITION suite enumerates *every* count vector summing to at most the stock, nothing pruned; Theorem 2’s 3-PARTITION suite enumerates every deployment fielding each type exactly once—the shape its witnesses take—with an unrestricted tier (arbitrary partial assignments) run under `-full` on four instances and the `hold` variant only. The searched fragment is thus a **proper subset** of the model’s play space—searched $\subsetneq$ model, never the whole thing—and the gap is closed by argument, not by search. Two actions of the full model are outside that fragment, and each is discharged explicitly rather than silently. *Pure movement and player DEFEND* are over-approximated: a stack that would move without attacking is instead allowed to *vanish from the board*, which frees strictly more hexes than any move could and therefore yields a sound upper bound. *WAIT*—which reorders terminal actions and leaves a blow at most nominal (Section 2.4)—is not searched on the player’s side; it is discharged on paper, per theorem: the no-directions of Theorems 1, 2 and 4 count blows and never refer to the acting order, and Theorem 3’s confinement lemma inducts over the realized order of terminal actions, which covers waiting stacks at their postponed position. The witness plays never wait. The *defence’s* waiting is not discharged but executed: the Theorem 1–2 suite and both Theorem 3 verifiers run a variant in which the enemy plays (§) literally, phase by phase, with the DEFEND bonus live in the damage formula—including the published `def 27` constants. One scope note, so this is not overread: since every non-waiting blow lands before any postponed DEFEND (Section 2.4) and the searched player never waits, no blow in these runs strikes a target that is already defending—instrumenting the damage calls confirms zero such calls.

What the (§) runs certify is the phase machinery and that every answer is invariant under it. The defended branch itself—`def 32, 0.025 · 31 = 0.775`, crossing the cap—is exercised arithmetically by `verify_mechanics.py` and by a phase-ordered trace in `test_regressions.py` in which a waiting player strikes a defended target at the capped multiplier; its engine reading is pinned by the harness’s ULP cases (Section 4.2). For the empirical instances of Section 5 the player-side gap is closed mechanically as well—see Section 5.1. This scope—which tiers are exhaustive over what, and on how many instances—is itemized suite by suite in Section 4.4 and in the proof documents; “every play” without qualification would overstate it.

3. **Cross-checking against the shipped engine.** A C++ harness links the actual VCMi battle classes—`DamageCalculator`, `CUnitState/CHealth`, `CRetaliations`, and the round-queue machinery `battleGetTurnOrder` and `battleQueuePhase`—and prints the numbers the engine itself produces. 49 cases; 46 exact agreements.

We are deliberate about the vocabulary. Nothing here is **machine-verified** in the sense of a proof assistant: there is no formal proof object. What we claim is that the constructions were **exhaustively regression-tested on bounded instances** and that the arithmetic was **engine-cross-checked**. Precisely what was cross-checked is: combat arithmetic including both damage factors and the clamp at 1, the health-pool and effective-count mechanics, the retaliation-charge mechanics—including 12 cases drawn from the reduction constructions—and the single-round turn queue under `WAIT/DEFEND` states. For that last item, the engine’s own `battleGetTurnOrder`, run over real `CUnitState` units, confirms four things: that the `NORMAL` phase descends in speed; that the `WAIT` phase follows it in *ascending* speed whatever the speeds (the engine half of the one-round lemma of Section 2.4, exercised on the exact speed-13-waits-against-speed-6 configuration that produced the Section 5.1 violations); that a defending unit leaves the queue for the round; and that the engine alternates sides on speed ties exactly where our reference model documents its simplified tie rule. Precisely what was *not* cross-checked: complete battles, obstacle boards, reachability and approach selection, and the `DEFEND` bonus’s server-side arithmetic and expiry (`BattleActionProcessor::doDefendAction`, `BattleInfo::nextTurn`), which cannot be driven without the full game server and are instead quoted from source with file-line citations and regression-tested in the reference model. No constructed instance has been played inside VCMi.

4.2 The three discrepancies, and a lesson about floating point

Three of the 49 engine cases disagreed with the model, all with defence far above attack. The cause is that VCMi’s hand-written JSON parser accumulates fractional digits as $0.1 \cdot d$ (R15, `lib/json/JsonParser.cpp:536-551`), so the literal 0.7—the defence-factor cap—is loaded as `0.7000000000000001`, one ULP above the correctly rounded double. The engine then computes $1 - 0.7000000000000001$, and `std::floor` then discards a point of damage whenever $\text{base} \times 0.3$ is an exact integer. The engine reports the constant itself as `defense_point_damage_factor_cap = 0.7000000000000001`.

The lesson is not that the engine is wrong. It is that a model which is “more correct” than the engine by one ULP still has to document the engine’s behaviour, and that a reduction whose arithmetic sits on such a boundary is fragile. Ours is positioned deliberately: Theorems 1, 2 and 4 set $\Delta = 0$ and never invoke the defence factor (for Theorem 1 this is a statement about the model’s exact integer arithmetic: its instances carry hit points in binary, and beyond 2^{53} —or the engine’s `int32` stack counts—the shipped engine

could not represent them at all, so no engine-agreement claim is made in that regime); Theorem 3’s undefended blows sit at $0.025 \cdot 26 = 0.65$, clear of the cap, while its defended branch (Section 2.4) does cross the cap and is exactly where the boundary bites—which is why the resource lemma is stated for every $\mu \in (0, 1)$: it holds on either side of the ULP.

4.3 The errors the checks caught

A verification method earns trust by what it catches, so we report what ours caught. The instructive one in full: the first version of Theorem 1 used an *attacking* garrison and claimed a slot of c_j creatures delivers c_j damage. It delivers $2c_j$ —when the enemy attacks, the player stack **retaliates**, striking that same enemy a second time in the same round—so the reduction decided SUBSET-SUM with budget $2B$, and the brute force reported yes-instances of the game for no-instances of PARTITION. The repair is the policy (§) of Section 2.4, and the same failure mode later resurfaced inside the checking code itself (error 3 below), which is why $R = 1$ together with the slowest possible defence does *not* make the defence’s policy irrelevant.

The other four, briefly (full accounts in the artifact’s VERIFICATION.md):

1. Theorem 2’s first version used a one-row battlefield, where a third stack cannot reach its enemy past an ally; caught by a geometry self-check on the *built* instance.
2. (The retaliation error above.)
3. The featureless verifier ran the attacking defence under both policy labels: 23 mismatches, three kills reported where the arithmetic admits one.
4. The empirical exact solver took one canonical approach hex per target instead of branching; the fix moved 5 of 858 responses, all upward.
5. In the first empirical run both prompt variants of an instance shared a batch and models copied their assisted answer into the raw variant; the raw condition was rerun in separate batches.

Errors 1–2 sat in proofs believed correct and written out in full; errors 3–5 sat in the checking apparatus itself. Verification code is code, so the suites carry a negative control: the single-type suite deliberately disables the resource lemma (enemy defence set equal to player attack) and confirms that a no-instance then *does* turn into a yes-instance—a pass carries evidence rather than silence.

Two further errors were caught not by the checks but by external review, and they mark the method’s boundary. The first is an earlier draft’s claim that a hit-point objective trivializes the problem, false by Corollary 4.2: a claim about instances nobody constructed, which bounded checks cannot see. The second is worse, because it was a claim about data we had. An earlier draft of Section 5.2 described the weak tier’s scores as declining with the slot count; the released per-response records plainly contradict it, and Section 5.2 now carries the numbers and the retraction. Every layer of this section checks *code against code*—a reduction against its source problem, a model against an engine, a bound against a witness—and that was a sentence in prose about what a table shows. The lesson we draw is not that the checking is worthless but that its scope is narrower than the volume of green output suggests: it certifies the theorems and the numbers, and says nothing about whether the sentences around them are true. A reader should weight the two differently, and so should we.

4.4 Reproducing

```
python3 scripts/verify_mechanics.py          # 90 checks: damage formula, WAIT/DEFEND phases
python3 scripts/test_obstacles.py            # 158 checks on geometry and blocking
python3 scripts/brute_force.py               # Theorems 1-2; defence variants incl. literal (§)
python3 scripts/dp_single_type.py            # Prop. 1.1: knapsack, corridors vs the game, control
python3 scripts/verify_featureless.py        # Theorem 4, tiers C1-C5
python3 scripts/verify_featureless.py --full # adds the exhaustive play tier C6
python3 scripts/verify_x3c.py                # Theorem 3, historical constants (def 41)
python3 scripts/verify_x3c.py --vacate       # ... admitting pure movement
python3 scripts/verify_x3c.py --defend       # ... with the defence playing (§) literally
python3 scripts/crosscheck_sol.py            # Theorem 3 as PUBLISHED (def 27, hp 4,  $\mu = 0.35$ )
python3 scripts/crosscheck_sol.py --defend   # ... published constants AND literal (§)
python3 scripts/search_free_order.py         # free activation order, vanish; both constant sets
python3 scripts/verify_embedding.py          # Lemma D.4's algorithm itself, on the whole corpus
python3 scripts/classify_skips.py            # classify every router skip (minutes, by hand)
python3 scripts/verify_hp_objective.py       # Corollary 4.2 (hit-point objective)
python3 empirics/scripts/verify_full_model_optima.py # Section 5 optima, full action model
python3 empirics/scripts/certify_scores.py   # Section 5 per-response scores, full action model
python3 empirics/scripts/check_defend_policy.py # (§) replay; --legacy-defend = control
python3 scripts/test_regressions.py         # regressions, incl. adapter figures vs code
cd engine-check && ./build.sh && ./run.sh && python3 compare.py
```

There are no dependencies beyond the Python standard library, except for the engine cross-check, which needs a VCMi checkout. Current outcomes are Table 2.

Table 2: What each suite runs, and its current outcome.

suite	scale	outcome
verify_mechanics.py	90 checks	all pass
brute_force.py	28 + 14 instances $\times$ 3 defence variants (hold, literal (§), attacking + NO_RETALIATION); the 3-PARTITION tier is $m = 2$ only, 11 yes / 3 no	all agree with PARTITION / 3-PARTITION
test_obstacles.py	158 checks	all pass
dp_single_type.py	2000 random instances against exhaustive knapsack search, 40 single-creature corridors played out exhaustively in the game model, plus a multi-creature negative control	agrees everywhere; the control disagrees, exactly as Proposition 1.1's hypotheses require
verify_featureless.py	46 instance runs, $m \in \{2, 3\}$, two defence variants	no mismatches
verify_x3c.py	31 instances (17 yes, 14 no), $q \leq 2$, historical constants (def 41, $\mu = 0.3$)	all agree, all winners canonical
verify_x3c.py -vacate	the same 31, pure movement admitted	identical answers
verify_x3c.py -defend	the same 31, defence executing (§) literally	identical answers
verify_x3c.py -full	55 instances (30 yes, 25 no), $q \leq 4$, 4 skipped by the router	all agree
-vacate		
crosscheck_sol.py	37 machine-built planar boards (23 yes, 14 no), 3 skipped by the router, under the published Theorem 3 constants (def 27, hp 4, $\mu = 0.35$)	all agree
-full		
crosscheck_sol.py	25 instances, published constants AND literal (§)—the combination	all agree
-defend	round 8 found had never been run	

suite	scale	outcome
<code>search_free_order.py</code>	16 boards — one corpus of eight $q = 2$ families, each built and searched under BOTH the historical (def 41) and the published (def 27) constants, the swap asserted at build time — searched with free activation order (any unacted player stack may act next), pass and vanish branches and exhaustive approach hexes, over every allocation (12 yes / 4 no builds), under the hold defence, which dominates (§) for the player	the free-order answer equals X3C and the fixed order on every (family, constant set, allocation) triple; yes-instances admit only the all-ones winner; 3 discriminating controls — order, vanish, destination — on which the full search strictly beats its degraded variants; the negative control (defence = attack) flips 1 planar no-instance to yes; the corpus tiers expanded 3860 out-of-order activations and 6312 vanish branches and met 0 states offering a second approach hex — Lemma D.7 from the searcher’s side, so the destination branch is exercised by its control only
<code>verify_embedding.py</code>	61 corpus families through the Lemma D.4 embedding algorithm itself (DMP planarity $\rightarrow$ orthogonal drawing $\rightarrow \lambda = 20$ scaling, adapters, stubs): 44 boards built; 17 certified degenerate no-instances, 3 of them also non-planar; 0 certified non-planar — plus a separate battery of 17 malformed encodings and one planted non-planar control, all deterministic	I1–I4, the Lemma D.4 feature counts ($4 C $ boxes, $6 C - X $ corridors) and the feature-based (SEP’) separation hold on every board (28151 non-incident feature pairs, class minima L_∞ 12/16/20 against the required 12); the no-certificate board, assembled from its own encoding, plays out as a genuine no; the full game search runs on 3 of them — under the historical AND the published constants — and agrees with X3C
<code>verify_hp_objective.py</code>	16 instances (10 yes, 6 no), plus a negative control	relaxation = mT iff 3-PARTITION; witness absorbs exactly mT
<code>verify_full_model_optima.py</code>	all 145 empirical instances	every optimum certified over the full action model; the ghost bound also dominates every (§) play (Section 5.1)
<code>certify_scores.py</code>	all 870 scored responses	every score certified likewise, and reproduced exactly by the (§) phase-aware replay
<code>check_defend_policy.py</code>	all 145 replayed under (§), phase-aware	all equal the recorded optima; -legacy-defend reproduces the 6 round-5 violations (negative control)
<code>test_regressions.py</code>	280 regressions: one per error ever caught here, a phase-ordered trace of the capped defended branch, and a doc-consistency battery that re-renders this very table from its manifest, re-runs the generating suites to pin the manifest’s counters (only the two -full Theorem 3 tiers excepted — those are swept against the proof document), sweeps the counter prose in the siblings and the paper body, and bans 19 retired claims verbatim	all pass
engine harness	49 cases	46 exact, 3 explained (Section 4.2); includes 6 turn-order cases run through the engine’s own <code>battleGetTurnOrder</code>

Table 2 is not maintained by hand: it is rendered into both versions of this paper from `verification_manifest.json` by `scripts/gen_verification_table.py`, and

`test_regressions.py` re-renders it in check mode, so an edit that bypasses the manifest fails the suite; the generator also validates each row against the manifest’s declared *ordered sequence* of counter placeholders and refuses digit-runs outside the row’s declared constants; and—because review round 11 demonstrated that a guard’s sentence about its own coverage is itself an unverified claim—the battery rebuilds eight demonstrated mutations (a counter retyped as a literal digit; two placeholders swapped inside one row; a counter moved across a cell boundary in one render only; a deleted duplicate digit-run; the (SEP’) class minima reordered; a sign placed in front of a literal; a digit-run deleted from the L^AT_EX render only; two constants swapped in the L^AT_EX render only) in memory on every run and requires each to fail validation. Each rendered into the papers under a green battery before the round that drilled it (11, 12 and 13); this sentence is backed by those drills rather than by intention. The guard’s reach stops at the numbers and their placement—ordered placeholder sequences and signed digit-runs, held cell by cell and in order across both renders after expanding the L^AT_EX macros that carry digits—and establishes no numeric *meaning*: the table’s *prose*—every non-numeric word of its claims—is rendered from the manifest but not generated from the artifacts, so a rewritten claim is a manifest edit the drills cannot see; review round 12 demonstrated exactly that, round 13 demonstrated it again (a transposed scale sentence, a swapped yes/no label), and we record the limit here rather than pretend otherwise. Since review round 9 the counters themselves are pinned to their artifacts rather than trusted as written: the battery re-runs the generating scripts—the mechanics tests, the obstacle, hp-objective and DP suites (the DP suite including its corridor game tier and multi-creature negative control), the Lemma D.4 embedding verifier, the featureless and brute-force sweeps, the legacy negative control, the Section 5.2 statistics generator and, since round 10, the default tiers of both Theorem 3 suites (roughly twenty seconds for `verify_x3c.py`, two and a half minutes for `crosscheck_sol.py`). It then compares each script’s own final output line with the manifest. The engine-harness verdicts are re-derived offline from the shipped engine outputs, and the empirical counters are the lengths of the shipped artifacts. One disclosed exception, so that a green run is not over-credited: the two `-full` corpus tiers—`verify_x3c.py -full -vacate` and `crosscheck_sol.py -full`, eight counters in all—are not re-run inside the battery (each takes minutes); those eight are swept against the sentences of the papers and the proof documents known to quote them, so a stale citation at a swept site fails the suite, but only rerunning the two tiers themselves re-derives the counts. Two further checks live outside the battery by policy, like the ability audit of Section 5: `scripts/check_artifact_repo.py`, which needs the network, and `scripts/fuzz_pipeline.py`, a seeded fuzz of the Lemma D.4 pipeline (300 random families through the router, every board checked for (I1)–(I4) and (SEP’), every no-certificate checked for truth) together with an exhaustive comparison of the DMP planarity test against an independent forbidden-subgraph oracle on all 32 768 labelled graphs on six vertices; both are clean at the cited tag.

One remark about the built/skipped split, so that rerunning the code holds no surprises. The board router for Theorem 3 is a randomized heuristic, and a skip is an honest failure of the layout search, never a board emitted in violation of the invariants. An earlier revision explained the skips by the typical non-planarity of random 3-uniform incidence structures; the per-family classification refutes that. There are 7 router skip events across the two full corpora—4 in the `-full -vacate` tier, 3 in the crosscheck corpus—concerning 5 distinct families (two are skipped in both corpora), and every one of them is a degenerate no-instance with an uncovered element, whose layout the hill-climb gives up on. Non-planarity does occur but is not what drives the skips: the corpora contain exactly 3

non-planar families, all of them also degenerate, and because the embedding algorithm certifies degeneracy *before* it tests planarity, they surface in the table’s degenerate count (“of which 3 are also non-planar”) while the “certified non-planar” count stays at zero—that zero records the order of the checks, not the absence of non-planar inputs (an earlier revision misread it exactly that way; the non-planar exit itself is exercised by a planted control). An earlier revision also cut the router’s retries with a per-instance wall-clock deadline, which made the built/skipped split depend on machine load; review round 8 caught the counts drifting between runs; the deadline is now gone (retries are capped by attempt count only), and the split is a deterministic function of the fixed seeds, so the numbers above reproduce exactly. What the suite asserts was never affected either way: every board it builds satisfies the invariants, every instance it builds agrees with the source problem, and on every yes-instance the unique winning allocation is the canonical one. A final transparency note: the corpus families are drawn by seeded random generators, and nothing here claims they are pairwise non-isomorphic—the instance counts measure runs of the pipeline, not distinct combinatorial structures.

Data and code availability. Everything this section runs is published as an artifact repository [Par26], release tag `v1.2`—cite and check out the tag, not the moving branch; round 12 of review caught the paper citing the unversioned repository while the public tree was one commit stale, and `scripts/check_artifact_repo.py` pins the tag to the paper—every path the paper names, whether in prose or in the commands above (round 13 widened that sweep from the 13 backticked paths to every file the paper names; their count is pinned in the manifest), the manifest counters, and a byte-identical `main.md`: the model transcription (`MODEL.md`), every script listed above together with the verification manifest that pins their outputs, the engine cross-check, the proof working documents, and the full empirical harness and response corpus of Section 5. All file paths in this paper are relative to the artifact root. The engine cross-check was run against a VCMi [VCMi26] checkout at commit `b5cee70`; every VCMi file this paper cites is byte-for-byte identical to the public tree at commit `deeab240` (`develop`, 2026-06-19), which is the anchor a reader should check out.

5 Empirical study: the optimum against one-shot model allocations

The theorems are asymptotic. This section asks a different and more concrete question: on instances small enough that the optimum can be computed exactly, how close to it do one-shot, model-proposed allocations come? To our knowledge no paper in this genre measures the gap between an exactly computed optimum and the decisions of contemporary models, which is why we include it. One framing rule first, because the word “agent” overpromises: **the measured object is the value of a model-proposed allocation under oracle-optimal tactical completion.** The models are one-shot allocation solvers here, not HoMM3-playing agents.

5.1 Design

The corpus is 145 instances of ARMY-ALLOCATION in four families, with slot counts $k \in \{2, \dots, 7\}$: a corridor family (60), a “flower” family with three deployment hexes per enemy (13), and two natural open-board families ($36 + 36$). The 72 natural instances take their creature statistics from the original game (extracted from `CRTRAITS.TXT`); the 73

corridor and flower instances are synthetic, built to mirror the reductions. Even in the natural families the statistics are **projected** rather than shipped verbatim, since H3-det needs flat damage: each creature’s damage range is collapsed to $dmg_{\min}$. Every instance has $R = 1$, no shooters and no double-wide creatures. The projection also drops every combat **ability**—CRTRAITS.TXT carries only the numeric block—so a stack labelled Devil carries a Devil’s numbers without BLOCKS_RETALIATION, a Crusader strikes once rather than twice, and a Psychic Elemental hits one target rather than every adjacent one. We count the affected creatures against VCMi’s `config/creatures/*.json` in three scopes, so that the disclosure cannot understate the projection by picking a flattering subset. Under the narrowest scope—the five abilities that would change the modelled melee arithmetic itself—47 of the 72 natural instances name at least one affected creature (BLOCKS_RETALIATION on 53 type-slots, ATTACKS_ALL_ADJACENT on 20, RETURN_AFTER_STRIKE on 13, FIRE_SHIELD on 8, ADDITIONAL_ATTACK on 5). Adding five more combat-adjacent abilities the projection also drops (SPELL_AFTER_ATTACK on 28, HP_REGENERATION on 19, MAGIC_RESISTANCE on 12, MANA_DRAIN on 5, LIFE_DRAIN on 2) raises that to 65. Under the widest scope—every shipped ability whatsoever (FLYING alone marks 128 type-slots, HATE 73)—all 72 are affected: no natural instance fields a creature exactly as the game ships it. The per-instance lists are `empirics/results/ability_projection.json`, generated by `audit_ability_projection.py`. The measurements are internally valid—both sides of every ratio are computed in the same fragment—but “statistics from the original game” means the numbers alone, not the creatures, and a certified optimum is the optimum of the *projected* instance: under the shipped abilities the optimal allocation could differ. How much it can differ is measured rather than conjectured: restoring a single dropped ability—the Efreet Sultan’s `hateGenies`, +50% against genies—moves the certified optimum of `naturalS-k6-02` from 1136 to 2020, so on that instance the projected optimum is 56% of the optimum of the instance its creature names describe (`empirics/scripts/check_ability_shift.py`, re-certified on both sides and pinned by the battery).

Every instance ships with a **certified** optimum, and the certificate is a pair rather than a single number: an upper bound from a reachability relaxation that dominates every legal play—including waiting, pure movement, and paths opened by enemy deaths—together with an allocation and a play attaining that bound in the simulator. An allocation achieving a value is on its own no evidence of optimality; it is the meeting of the two that is (`empirics/scripts/verify_full_model_optima.py`, all 145 pass, and `empirics/scripts/certify_scores.py`, which does the same with the allocation held fixed and certifies all 870 per-response values).

A discrepancy found by external review, and how it was closed. The corpus was generated with a scripted defence that **takes no action at all** (`enemy_policy: "hold"`), while an earlier version of the paper’s policy (§) had every enemy issue DEFEND at its own turn. The two differ exactly when an enemy acts before the player stack that strikes it—false for all 73 corridor and flower instances, true on 53 of the 72 natural ones—and the difference was not cosmetic: under DEFEND-at-turn, **six of the 145 recorded optima were not attainable at all**—`naturalM-k3-04` (recorded 1722 against a generous upper bound of 1564), `naturalS-k2-05` (867 vs 624), `naturalS-k5-05` (3111 vs 2826), `naturalS-k6-01` (285 vs 190), `naturalS-k7-01` (4311 vs 3895), `naturalS-k7-03` (2136 vs 1068). In `naturalS-k2-05`, for instance, the Dragon Flies have speed 13 against the player’s speed 6, so they defend before the player’s blow, their defence rises from 10 to 12, and the blow the corpus records

as killing ($\lfloor 5 \cdot 4 \rfloor = 20$ against 20 hit points) delivers $\lfloor 5 \cdot 4 \cdot 0.95 \rfloor = 19$ instead. The resolution is the policy Section 2.4 now defines: ($\dagger$) waits, then defends, so every enemy’s DEFEND lands in the WAIT phase—after every non-waiting player blow, regardless of speeds. Under this ($\dagger$) the recorded numbers are exact, and the claim is machine-checked from both sides rather than argued. The witness side: `empirics/scripts/check_defend_policy.py` replays every recorded optimal allocation through the attack-only play search in a phase-aware simulation whose defence executes ($\dagger$) literally—WAIT at the NORMAL activation, DEFEND, with the bonus live in the damage formula, at the postponed one—and requires exact equality with the recorded optimum: 145 of 145 pass, and `empirics/scripts/certify_scores.py` applies the same replay to all 870 scored responses. The bound side: the ghost-reach relaxation of `empirics/scripts/verify_full_model_optima.py` prices every blow at nominal damage and dominates every legal play, so it upper-bounds every ($\dagger$) play as well. As a negative control, the same replay under the old DEFEND-at-turn policy (`check_defend_policy.py -legacy-defend`) reproduces all six violations above—plus eight more, the exact replay being sharper than round 5’s generous bound—which is the evidence that the phase machinery is live rather than vacuously green. The optima, baselines and responses below therefore stand as ($\dagger$) numbers, not as numbers of a scripted-defence variant; the theorems never depended on this corpus either way.

The model’s task is the pre-battle allocation only. After it commits, the battle is played out by exhaustive search over attack-only plays—each stack passes or performs WALK_AND_ATTACK, branching over targets and over every legal approach hex. This is deliberately generous: we do not want to penalize a model for tactics it did not choose. As with the optima, the search omits WAIT and MOVE-only actions, so the value it credits to an allocation is not on its face an optimum over the whole action model, and a referee is right to ask whether the published ratios are therefore understated. They are not: the same relaxation closes with the allocation held fixed instead of quantified over, and all 870 scored responses meet that upper bound (`empirics/scripts/certify_scores.py`, all pass)—and, as described above, the same script certifies each response’s value under ($\dagger$) by phase-aware replay, so numerator and denominator alike are ($\dagger$) numbers. Each instance appears in two prompt variants: **assisted**, where the prompt lists which enemy stacks each slot can reach, and **raw**, where it gives only coordinates and the distance rule. The headline uses **assisted**, because the claim under test is about the allocation decision, not about hex arithmetic.

Three tiers of one model family were run—`claude-haiku-4-5`, `claude-sonnet-5`, `claude-opus-5`—**one completion per cell**, no tools, no simulator access, identical sampling settings across tiers. A conflict note: the same model family assisted in writing this paper (the Acknowledgements name the models); the subjects here ran one-shot, without tools, and saw only the task prompts. One provenance limitation is flagged rather than hidden: the version history preserves every response byte-for-byte and every score is recomputed from those artifacts, but no per-call parameter manifests survive, so the sampling conditions themselves (temperature, one-shot discipline) rest on the run protocol, not on a recorded artifact. One completion per cell means every number below describes a single realized run: there are no repeats, and therefore no uncertainty intervals; we draw no conclusions that would need them. The full matrix is 870 cells (3 models $\times$ 145 instances $\times$ 2 variants) and is complete: tasks were administered in batches of ten, and 12 initially missing cells (token-limit truncations concentrated on hard `sonnet` raw cases, plus two lost

responses) were filled by separate single-task runs rather than dropped—an easier protocol, which is a heterogeneity we flag rather than hide; the refill lowered one figure (`sonnet` raw, $0.992 \rightarrow 0.991$) and is reported as such. Both stages of the response set—the 858-cell batched run and the completed 870-cell matrix—are preserved in the artifact’s version history with the twelve refilled cells identifiable exactly, so the shift is reproducible, not anecdotal. Responses were scored against the certified optima; a response that violates the stock budget is invalid and scores zero, since the budget is the constraint that makes the problem a knapsack. All three scorer strictness columns (`strict`, `valid_only`, `repaired`) agree to three decimals, and in the final matrix every one of the 870 responses parsed and was budget-legal, so the treatment of invalid responses drives nothing.

5.2 Results

Table 3 gives the ratio to the certified optimum, and the fraction of instances solved exactly:

player	assisted	exact	raw	exact
<code>claude-haiku-4-5</code>	0.926	76.6 %	0.881	74.5 %
<code>claude-sonnet-5</code>	0.997	97.9 %	0.991	97.9 %
<code>claude-opus-5</code>	0.998	97.2 %	0.997	97.9 %
greedy-value heuristic	0.954	79 %	—	—
greedy-density heuristic	0.899	57 %	—	—
100-sample random search baseline	0.971	—	—	—

Table 3: Model tiers and non-learned baselines on the 145-instance corpus.

(The last row is a *search* baseline—the best of 100 random legal allocations per instance—so it consumes 100 attempts where each model consumes one; it is a yardstick, not a like-for-like competitor.)

Three observations, all descriptive of this single run.

1. **On this run the gap sits between the weakest tier and the mid tier, not between the mid tier and the frontier.** Sonnet and Opus score within 0.001 of each other—a difference of about one instance, well within what run-to-run noise or a ceiling effect could produce, so we read it as “similar on this benchmark”, not as evidence about saturation. Haiku loses 7.4 % of the available value and misses the optimum on roughly one instance in four—worse than the better of the two greedy heuristics.
2. **The weak tier is below the strong tiers at every slot count, but its scores do not decline with the slot count.** Every k bucket, assisted:

k	2	3	4	5	6	7
<code>claude-haiku-4-5</code>	0.981	0.973	0.857	0.946	0.952	0.828
<code>claude-sonnet-5</code>	1.000	1.000	1.000	0.993	1.000	0.990
<code>claude-opus-5</code>	1.000	1.000	1.000	0.993	0.998	0.995

Haiku’s worst buckets are $k = 7$ and $k = 4$ and its best is $k = 2$, but the sequence is **not** monotone—it rebounds at $k = 5$ and $k = 6$ —and the per-response rank

correlation between k and the ratio is only weakly negative (Spearman -0.19 , and -0.25 on the natural families alone)—a modest association, not the strong ordering suggested by quoting $k = 2, 4$ and 7 alone. Two confounds compound this: the family mix changes with k , since the flower family exists only at $k \in \{3, 6\}$ and contributes ten extra instances at $k = 6$, and the natural instances at different k draw unrelated creatures and budgets. Restricting to the two natural families, the only ones present at every k , does not restore a decline either $(0.965, 0.943, 0.737, 0.902, 0.915, 0.701)$.

An earlier draft of this paper claimed monotone degradation in k and called it the study’s most informative figure. That claim was false, and it was reached by quoting $k = 2, 4, 7$ and passing over the rebound. We retract it. **We assert no scaling law in k .** What survives is the tier separation of observation 1, which does not involve k . Sonnet and Opus stay above 0.985 at every k .

3. **Supplied geometry helped mainly the weak tier.** Removing reachability from the prompt cost Haiku 4.5 percentage points, Sonnet 0.61, and Opus 0.027 on this run (unrounded, Opus scores 0.9976 assisted against 0.9973 raw)—the Opus figure amounts to about one instance and is therefore not a measurement at one completion per cell. (These are the unrounded gaps as `stats_recheck.py` prints them; an earlier draft quoted 0.6 and 0.03, rounded from two different provenances.)

5.3 Limitations, in the text and not in a footnote

The instances are small—they have to be, since the optimum must be computed exactly—and the 100-sample random baseline already reaches 0.971 . The defensible claim is therefore “on this benchmark, one-shot weak-tier allocations systematically fell short of an exactly known optimum”, **not** “language models cannot solve an NP-hard problem”, and **not** any claim about how the shortfall scales with k —observation 2 retracts that. The strong tiers in fact solve these small instances; the difficulty the theorems describe lives in the asymptotics, and this study illustrates that rather than contradicting it. The design limits are worth listing plainly: one completion per cell, so every comparison is descriptive of a single run and differences of a few thousandths are not evidence of anything; the protocol is heterogeneous (858 cells in batches of ten, 12 refills in single-task contexts, and the refills sat non-randomly on hard cases); the k -trend is confounded with family composition (observation 2); only one model family was tested, so a cross-vendor comparison is future work; and reinforcement-learning agents were not measured, because neither of the two HoMM3 RL environments we examined can express a pre-battle allocation at all. A publication-grade version of this study would use fresh single-task contexts throughout, repeated calls per cell with paired intervals, a balanced generator, and at least one more model family; until then this section claims description, not inference.

6 Related work

Turn-based tactics. Gao [Gao19] gives the only prior complexity study of a commercial turn-based tactical RPG, proving a simplified Fire Emblem PSPACE-complete and its round-bounded version NP-complete. Two features matter for the comparison: every numeric attribute in that model is a constant bounded by 8—so the hardness is purely combinatorial, and the NP-completeness trivially strong—and the player’s army is *given*, with no allocation decision anywhere. That hardness comes from geometry; ours (in Theorems 1, 2 and 4) comes from arithmetic.

Attrition games. The closest prior result is Furtak and Buro [FB10], who study attrition games on graphs—nodes are units with $\langle \text{health}, \text{attack} \rangle$, edges say who may attack whom, movement removed. Their Theorem 7 shows that *attack partitioning*, in which a unit divides its attack power among its targets, is NP-hard by reduction from SUBSET-SUM; their Theorem 8 partitions a set of attacking units against health thresholds. **These are the mechanisms of our Theorems 1 and 2, published in 2010, and we do not claim them as new.** Our claims against [FB10] are narrower: the resource we split is a native player-facing action of a shipped game rather than a modelling device introduced to interpolate between models; it is a discrete count of creatures rather than a divisible scalar; the map from allocation to reachable enemy is realized by board geometry rather than supplied as an abstract graph—though, as Section 2.1 concedes, we choose the deployment cells, so this map is not *forced* by the shipped formation tables either; and our hardness is strong, whereas every numeric hardness result in [FB10] is weak. A systematic sweep of AIIDE (2005–2025, workshops included) and CIG/CoG (2008–2025)—roughly 2900 titles over two independent passes—found no other hardness result about combat or force allocation at either venue. [FB10] closes with the question our Theorem 1 sits inside: what is the smallest k for which the k vs. n problem is NP-hard?

Card games. Hoffmann, Lynch and Winslow [HLW20] reduce 3-PARTITION to board-scaled *Lethal* in Hearthstone: $3n$ minions with attack $4a_i$ against n taunts of health $4S/n$, no overkill possible, winning assignments exactly the 3-partitions. **This is the closest published relative of our Theorems 2 and 4, in a shipped commercial game, six years earlier.** Their Theorem 3.1 states NP-hardness; since the reduction is from 3-PARTITION on a board-scaled instance it is strong, and strong NP-hardness as such is not new in this genre—we do not claim it is. Bosboom and Hoffmann [BH17] prove Netrunner mate-in-1 weakly NP-hard from 2-PARTITION; Romão et al. [RPU25] model an aggressive Flesh and Blood turn as an ILP containing 0-1 Knapsack, obtaining a weak hardness result that the authors themselves expect to be pseudo-polynomially solvable. UNO is NP-hard for a single player [DDHUU14]; perfect-information Hearthstone is PSPACE-hard [Zha23]; Magic: The Gathering is Turing-complete [CBH19].

Classical ancestry. With a single type, $R = 1$, and each slot facing one enemy, our problem is 0-1 Knapsack [Kar72] and, with $v_j = a_j$, PARTITION [GJ79, SP12]. With unit-multiplicity heterogeneous attackers and per-slot thresholds it is a form of BIN COVERING, strongly NP-hard by Assmann, Johnson, Kleitman and Leung [AJKL84]. Weapon-Target Assignment is NP-complete [LW86] but draws its hardness from a nonlinear probabilistic objective, so it is an ancestor rather than a competitor. The Colonel Blotto literature runs the other way, with polynomial equilibrium algorithms including the indivisible case [Har08]; hardness appears only when the resource stops being homogeneous [DSST21]. Stripped of the game, Theorem 4 *is* bin covering, and we say so: the interest is in how little of HoMM3 the hardness needs, not in the combinatorial core.

Composition rather than allocation. Ponomarenko and Sirotkin [PS20] prove optimal team choice in the auto-battler Dota Underlords NP-complete. That is the exact complement of our framing: there the difficulty is entirely in *which* units to field and the arrangement is free; in our Theorem 1 there is nothing to choose and the difficulty is entirely in the split.

HoMM3. There is no prior complexity result for any title in the Heroes of Might and Magic, King’s Bounty, Disciples, Age of Wonders or Master of Magic series. We are aware of two academic papers on HoMM3, neither about complexity: Diochnos [Dio10] models the random secondary-skill offers of levelling up; Kowalski et al. [KMPPPS18] generate balanced maps from terrain features.

7 Open problems

We list these in the order we would attack them, and we deliberately defer several that a reader might expect.

1. **A natural victory objective.** All four theorems use the artificial deadline $R = 1$ and the objective “value destroyed”. Replacing it by “eventually eliminate the defence” is the most valuable single improvement. A promising route makes every item creature a shooter with one shot, separated from the defence by an impassable barrier, so that surviving enemies can never be damaged after the first volley; it needs shooting and ammunition in the executable model, which the formal model already retains.
2. **Fixed k .** The shipped game has $k = 7$; all four constructions need k to grow. We conjecture the problem is in P for fixed k and $R = 1$. Proving it would give the contrast result the separation wants.
3. **Native deployment formations.** All four theorems take the slot-to-hex map as input (Section 2.1). Recovering any of them when deployment comes from the engine’s formation tables is open, and is the sharpest form of the objection “your board is not a game position”.
4. **Single type on a featureless board.** Theorem 3 needs obstacles; Theorem 4 needs a diverse roster. Whether complete reachability plus a single type is tractable is open even with one creature per enemy stack: the obvious cardinality-constrained knapsack is only an upper bound, because its achievability needs the chosen stacks to hold seats at distinct enemies *simultaneously*, and complete reachability—a property of the starting position—does not supply that (a six-hex instance in the executable model has knapsack value 2 and true optimum 1; Theorem 4 closes the same gap with the routing argument of Appendix E). Enemy stacks of many creatures, where partial kills score, may be harder still, and that is the version a referee is most likely to ask for.
5. **Bounded speed.** Theorem 3 sets creature speed to the board size to avoid equalizing corridor lengths. Legal, but inelegant.
6. **Adversarial defence.** Replacing the scripted policy by an optimizing opponent moves the problem out of NP. The metatheorems of de Haan and Wolf [dHW18] suggest the second level of the polynomial hierarchy rather than PSPACE if one player is strategically restricted.
7. **Approximation.** None of our results gives inapproximability. Since Theorem 4 is bin covering, positive approximation results would have to beat what is already known offline, so this is less attractive than it looks.

8 Conclusion

Planning one round of HoMM3 combat is hard in two separable ways: choosing how to divide an army among slots, and choosing what to hit. Each remains strongly NP-hard under conditions that remove the other—a fixed single-type army for the second, a board that dictates the targeting for the first—and the tractable case we exhibit is tractable because its reach structure is a perfect matching over single-creature stacks, not because its roster is poor. Every rule used is cited to a line of an engine that reproduces the shipped game’s combat, and every reduction has been regression-tested on bounded instances against an executable transcription of those rules, to the scope itemized in Section 4—a process that changed two of the proofs in this paper.

Acknowledgements

The proofs, the machine-checking apparatus and the drafts of this paper were produced with substantial AI assistance: several large language models worked as separate agents on the model transcription, proof exploration and drafting, the verification code, the literature survey and the empirical harness. Concretely: the model transcription, the proofs, the verification code and the drafts were written by Anthropic’s Claude agents (Claude Fable 5 and Claude Opus 5, run through Claude Code); the second, independent proof of Theorem 3 by an OpenAI GPT-5.6 agent (`codex`); and the adversarial review rounds by OpenAI GPT-5.6 (`codex CLI`), DeepSeek v4-pro, and Claude Fable 5 and Claude Opus 5 agents. Theorem 3 was proved twice, independently and in parallel, by two agents that were not permitted to see each other’s work; the two proofs agreed and were merged. The author treats that agreement as an error-catching redundancy, not as independent scientific validation. The mechanics and the constructions were tested to the scope stated precisely in Section 4—bounded instances and selected engine mechanics, not all statements and not a formal proof object. Every reference cited was independently located; the depth to which each load-bearing statement from the literature was verified varies by reference and is recorded in Appendix A—the two dependencies of Theorem 3 in detail, and every citation not read in full listed explicitly. The author reviewed the text and is responsible for the content.

References

- [AJKL84] S. F. Assmann, D. S. Johnson, D. J. Kleitman, J. Y.-T. Leung. On a dual version of the one-dimensional bin packing problem. *Journal of Algorithms* 5(4):502–525, 1984. [https://doi.org/10.1016/0196-6774\(84\)90004-X](https://doi.org/10.1016/0196-6774(84)90004-X)
- [BH17] J. Bosboom, M. Hoffmann. Netrunner Mate-in-1 or -2 is Weakly NP-Hard. arXiv:1710.05121, 2017. <https://arxiv.org/abs/1710.05121>
- [CBH19] A. Churchill, S. Biderman, A. Herrick. Magic: The Gathering is Turing Complete. arXiv:1904.09828, 2019; FUN 2021, LIPIcs 157, art. 9, pp. 9:1–9:19. <https://doi.org/10.4230/LIPIcs.FUN.2021.9>
- [DDHUUU14] E. D. Demaine, M. L. Demaine, N. J. A. Harvey, R. Uehara, T. Uno, Y. Uno. UNO is hard, even for a single player. *Theoretical Computer Science* 521:51–61, 2014. <https://doi.org/10.1016/j.tcs.2013.11.023>

- [DF86] M. E. Dyer, A. M. Frieze. Planar 3DM is NP-complete. *Journal of Algorithms* 7(2):174–184, 1986. [https://doi.org/10.1016/0196-6774\(86\)90002-7](https://doi.org/10.1016/0196-6774(86)90002-7)
- [DG13] C. A. Duncan, M. T. Goodrich. Planar Orthogonal and Polyline Drawing Algorithms. In R. Tamassia (ed.), *Handbook of Graph Drawing and Visualization*, ch. 7, pp. 238–261, CRC Press, 2013. <https://doi.org/10.1201/b15385-10>
- [dHW18] R. de Haan, P. Wolf. Restricted Power — Computational Complexity Results for Strategic Defense Games. FUN 2018, LIPIcs 100, art. 17, pp. 17:1–17:14. <https://doi.org/10.4230/LIPIcs.FUN.2018.17>
- [Dio10] D. I. Diochnos. Leveling-Up in Heroes of Might and Magic III. FUN 2010, LNCS 6099, pp. 145–155. https://doi.org/10.1007/978-3-642-13122-6_16
- [DSST21] S. Dehghani, H. Saleh, S. Seddighin, S.-H. Teng. Computational Analyses of the Electoral College. AAAI 2021, pp. 5294–5302. <https://doi.org/10.1609/aaai.v35i6.16668>
- [FB10] T. Furtak, M. Buro. On the Complexity of Two-Player Attrition Games Played on Graphs. AIIDE 2010, pp. 113–119. <https://doi.org/10.1609/aiide.v6i1.12410>
- [Gao19] J. Gao. The Computational Complexity of Fire Emblem Series and similar Tactical Role-Playing Games. arXiv:1909.07816, 2019. <https://arxiv.org/abs/1909.07816>
- [GJ79] M. R. Garey, D. S. Johnson. *Computers and Intractability*. Freeman, 1979.
- [Har08] S. Hart. Discrete Colonel Blotto and General Lotto games. *International Journal of Game Theory* 36(3–4):441–460, 2008. <https://doi.org/10.1007/s00182-007-0099-9>
- [HLW20] M. Hoffmann, J. Lynch, A. Winslow. Mad Science is Provably Hard: Puzzles in Hearthstone’s Boomsday Lab are NP-hard. arXiv:2010.08862, 2020. <https://arxiv.org/abs/2010.08862>
- [Kar72] R. M. Karp. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations*, Plenum Press, 1972, pp. 85–103. https://doi.org/10.1007/978-1-4684-2001-2_9
- [KMPPPS18] R. Kowalski, M. Miernik, M. Pytlik, M. Pawlikowski, K. Piecuch, J. Sękowski. Strategic Features and Terrain Generation for Balanced Heroes of Might and Magic III Maps. IEEE CIG 2018, pp. 1–8. <https://doi.org/10.1109/CIG.2018.8490430>
- [LW86] S. P. Lloyd, H. S. Witsenhausen. Weapons allocation is NP-complete. *Proc. 1986 Summer Conference on Simulation*, 1986.
- [Par26] I. Parfenchuk. Artifact repository for this paper: model transcription, proofs, verification scripts, engine cross-check and empirical harness. <https://github.com/uson1x/homm3-hardness>, release tag v1.2, 2026.

- [PS20] A. A. Ponomarenko, D. V. Sirotkin. Dota Underlords game is NP-complete. arXiv:2007.05020, 2020. <https://arxiv.org/abs/2007.05020>
- [RPU25] L. G. Romão, S. P. de Paula, E. T. Ueda. Optimizing for aggressive-style strategies in Flesh and Blood is NP-hard. arXiv:2501.11683, 2025. <https://arxiv.org/abs/2501.11683>
- [TT89] R. Tamassia, I. G. Tollis. Planar grid embedding in linear time. *IEEE Transactions on Circuits and Systems* 36(9):1230–1234, 1989. <https://doi.org/10.1109/31.34669>
- [VCMi26] The VCMi Project. VCMi: open-source engine reimplement of Heroes of Might and Magic III. <https://github.com/vcmi/vcmi>, GPLv2. Cited files anchored at public commit `deeab240` (develop branch, 2026-06-19).
- [Zha23] Z. Zhang. Perfect Information Hearthstone is PSPACE-hard. arXiv:2305.12731, 2023. <https://arxiv.org/abs/2305.12731>

A Status of the citations

The two literature dependencies of Theorem 3 were checked on 2026-08-03; here is exactly what was verified and from what.

- [DF86]—**read in full against the published paper** (scan of *J. Algorithms* 7:174–184). Planarity is defined on the triple/element incidence graph: “We have a vertex for each element ... and each triple ... There is an edge connecting a triple to an element if and only if the element is a member of the triple. ... We will say that the instance is *planar* if G is planar” (p. 175). **Lemma 2.2 states “Planar X3C is NP-complete” outright**, proved directly from Planar 1-3SAT, so the earlier “Planar 3DM, hence Planar X3C by identity” detour is unnecessary and has been removed. Bonus, from p. 178: their X3C instances have every element in two or three sets, so PLANAR-X3C remains NP-complete under that degree restriction.
- [TT89]—the exact statement is quoted from an authoritative secondary source, the *Handbook of Graph Drawing and Visualization* chapter on orthogonal drawings [DG13] (hosted by Tamassia), whose Theorem 7.3 credits [TT89] with: a biconnected 4-plane graph admits an orthogonal grid drawing in $O(n^2)$ area with at most $2n + 4$ bends, and **a connected 4-plane graph, with at most $2.4n + 2$ bends and no edge bending more than four times**. Theorem 7.3 itself states no running time; the linear-time claim is the title of [TT89] and the chapter’s framing text, and is corroborated by the primary abstract quoted below. The argument uses only (D5), polynomial time. The connected case is the one Lemma D.4 uses (its graph is connected after step 2, with maximum degree 3). The abstract of the IEEE original was additionally obtained as recorded in a bibliographic index (the OpenAlex record, reconstructed from the record’s inverted word index—not the publisher’s own text) and states verbatim: an $O(n)$ -time algorithm producing grid embeddings with “the total number of bends is at most $2.4n+2$ ”, “the number of bends along each edge is at most 4”, edge length $O(n)$, and area $O(n^2)$ —every number the lemma relies on thus matches an independent record of the primary abstract. The abstract does not state the connectivity hypothesis, so the **connected-case theorem itself—the statement Lemma D.4 uses—still rests on the Handbook alone** (the

constant $2.4n + 2$ is incidental: the proof never uses it); the paywalled proof body and the biconnected $2n + 4$ refinement (not used here) remain unread.

Everything else cited was read in full, with three exceptions, listed so that the previous sentence cannot silently overclaim: [HLW20] was read in detail in its sections 1–3 and its load-bearing theorem statements were transcribed verbatim, but not cover to cover; [PS20] was verified from its abstract and problem statement; [BH17] from its abstract and introduction. All three are cited as related work—for which game they study and that a hardness result exists—and none of their constructions is reused anywhere in this paper.

B Rule-to-source table

Table 4 gives every rule tag used in the body, with its `file:line` in the VCMi checkout at commit `b5cee70`. Paths are relative to the repository root; `lib/battle/` is abbreviated `lb/` and `server/battles/` is abbreviated `sb/`.

Table 4: Rule tags, their statements, and their sources in the VCMi checkout.

tag	rule	source
R1	hex grid, offset coordinates, six neighbour directions	lb/BattleHex.h:60–73, 147–178
R2	hex distance via axial coordinates	lb/BattleHex.h:195–210
R3	impassable hexes: battlefield set, obstacle objects, siege walls	lb/CBattleInfoCallback.cpp:1328–1391
R4	a living unit blocks its hex; a dead one does not	lb/CBattleInfoCallback.cpp:1355–1360
R5	stack health pool (<code>fullUnits</code> , <code>firstHPleft</code>)	lb/CUnitState.cpp:183–186
R6	effective count <code>fullUnits + [firstHPleft > 0]</code>	lb/CUnitState.cpp:282–285
R7	damage: attack/defence factors and the lower clamp at 1	lb/DamageCalculator.cpp:210–224, 322–337, 123–131, 576–577
R8	kill threshold; overkill discarded	lb/DamageCalculator.cpp:522–531; lb/CUnitState.cpp:202–203
R9	turn order: speed desc., side, slot; WAIT phase after NORMAL, ascending speed	lb/BattleInfo.cpp:978–1006; lb/CBattleInfoCallback.cpp:496–509, 601–623; phase enum lb/Unit.h:33–43
R10	speed doubles as initiative and movement range	lb/CUnitState.cpp:589–600
R11	movement: unweighted BFS over enterable hexes; WALK_AND_ATTACK	lb/CBattleInfoCallback.cpp:1411–1469; sb/BattleActionProcessor.cpp:216–352
R12	retaliation after the blow; once per round; the dead do not retaliate	sb/BattleActionProcessor.cpp:298–334; lb/CUnitState.cpp:484–490
R13	DEFEND: ends turn, +20 % defence (floor +1) until next turn	sb/BattleActionProcessor.cpp:160–212, 693; turn passes on: sb/BattleFlowProcessor.cpp:804–868
R14	shipped bounds: 11×17 board, ≤ 7 slots, formation tables for deployment	lb/BattleHex.h:19–24; lib/constants/NumericConstants.h:32; config/gameConfig.json:625, 635–643, 653–661
R15	JSON parser loads the literal 0.7 as 0.7000000000000001	lib/json/JsonParser.cpp:536–551

tag	rule	source
R16	first and last columns are not usable cells (SIDE_COLUMN)	lb/BattleHex.h:97-100; lb/CBattleInfoCallback.cpp:1321-1326
R17	equal-initiative ties alternate sides by sideThatLastMoved; attacker priority only on the first turn	lb/CBattleInfoCallback.cpp:474-509

C The four adapter patterns of Lemma D.3

An orthogonal drawing delivers the three edges of a set-vertex along three of the four axis directions; the three dockings are the alternating triple $\{\text{TOP_LEFT}, \text{RIGHT}, \text{BOTTOM_LEFT}\}$. The gap is bridged by a 9×9 box, one pattern per missing axis direction. In the local frame the enemy Z sits at $(4, 4)$ with row 4 even; ports (where a corridor crosses the box boundary) are the four boundary midpoints. Notation: # impassable, . free corridor hex, Z the enemy, and $U = (4, 3)$, $R = (5, 4)$, $D = (4, 5)$ the three dockings; even rows are drawn half a step to the right, as on the hex board—from an even row the two upper neighbours sit at columns x and $x + 1$ (rule R1). Each arm is a path from its port to its docking; no two arms share or touch a hex; the arm touches Z only at the docking; the unused port stays impassable. All four patterns are machine-checked hex by hex (`verify_x3c.py::check_enemy_adapters`).

Missing LEFT (all three arms straight):

```
# # # # . # # # #
# # # # . # # # #
# # # # . # # # #
# # # # U # # # #
# # # # Z R . . .
# # # # D # # # #
# # # # . # # # #
# # # # . # # # #
# # # # . # # # #
```

Missing RIGHT (the left arm dips one row to reach D; the bottom arm swings around the box to reach R, preserving the cyclic order):

```
# # # # . # # # #
# # # # . # # # #
# # # # . # # # #
# # # # U # # # #
. . . # Z R . # #
# # . . D # . # #
# # # # # # . # #
# # # # . . . # #
# # # # . # # # #
```

Missing BOTTOM (left arm dips to D; top and right arms straight):

```
# # # # . # # # #
# # # # . # # # #
```

```

# # # # . # # # #
# # # # U # # # #
. . . # Z R . . .
# # . . D # # # #
# # # # # # # # #
# # # # # # # # #
# # # # # # # # #

```

Missing TOP (left arm rises to U; right and bottom arms straight):

```

# # # # # # # # #
# # # # # # # # #
# # # # # # # # #
# # . . U # # # #
. . . # Z R . . .
# # # # D # # # #
# # # # . # # # #
# # # # . # # # #
# # # # . # # # #

```

D The full proof of Theorem 3

This appendix makes Theorem 3 and Corollary 3.1 self-contained: the complete construction, the embedding lemma with its proof, and the correctness argument, expanding the outline of Section 3.3. The statements of Theorem 3, Corollary 3.1 and the resource lemma (Lemma 3.2), the four invariants (I1)–(I4), and the enemy-gadget picture are in Section 3.3 and are not repeated; the adapter patterns are printed in Appendix C. Citation provenance for the two literature dependencies—Dyer–Frieze [DF86] and Tamassia–Tollis [TT89] via [DG13]—is Appendix A.

D.1 The instance

Given a PLANAR-X3C instance (X, C) with $|X| = 3q$ —a universe X , a collection C of 3-element subsets whose bipartite incidence graph $G(X, C)$ is planar, asking for q pairwise disjoint members covering X —build the ARMY-ALLOCATION instance $G_3(X, C)$:

- **Player army.** One creature type P with $att = 1$, $def = 1$, flat damage 1, $hp = 4$, $spd = \sigma$ (the board’s hex count), $value = 0$, and stock exactly $3q$. There are $k = 3q$ slots, one per element $e \in X$, at deployment hexes p_e .
- **Defence.** For each $S \in C$, one stack E_S of **one** creature of type Q with $att = 1$, $def = 27$, flat damage 1, $hp = 3$, $spd = 1$, $value = 1$, at hex z_S , playing the policy (‡) of Section 2.4.
- **Board.** Produced by Lemma D.4 below, satisfying (I1)–(I4). **Question:** $R = 1$, $W = q$.

With $\Delta = att(P) - def(Q) = -26$ and $0.025 \cdot 26 = 0.65 < 0.7$, an *undefended* blow never reads the cap constant, and a stack of c player creatures delivers nominal damage $D(c) = \max(1, \lfloor 0.35c \rfloor)$ (Section 3.3); the defended branch does read it—see below. Every numeric parameter is a constant except σ , the stock $3q$ and $W = q$, all bounded by

a polynomial in $|X| + |C|$, so the instance is polynomial even under **unary** encoding—which is what makes the hardness strong. The choice $def = 27$ is not delicate: any $def(Q) > att(P)$ gives $\mu \in (0, 1)$, which is all Lemma 3.2 needs; 27 merely keeps 0.65 clear of the floating-point hazard of Section 4.2 on both sides. A *defending* E_S has defence $27 + \lfloor 27 \cdot 20/100 \rfloor = 32$, so $\Delta = -31$, $0.025 \cdot 31 = 0.775$ is past the cap and μ drops from 0.35 to 0.3—still in $(0, 1)$, so Lemma 3.2 applies unchanged to the only blows the bonus can ever touch (a waiting player’s, Section 2.4).

D.2 Geometry from the invariants

Lemma D.1 (Reach). *Assume (I1)–(I4), and let a position be reached in which every E_S with $S \ni e$ is alive and R_e holds exactly one player stack, namely slot e ’s. Then that stack can strike exactly the enemies E_S with $S \ni e$, and for each such S its only legal approach hex is the docking d_S^e . The starting position is the special case.*

Proof. Every hex outside R_e that is adjacent to R_e is either impassable or holds a living enemy, and neither is enterable (R3, R4): by (I2) the regions are the connected components of the free hexes once the enemy hexes are removed, so the only free non-region neighbours of R_e are enemy hexes, and by (I3) those are exactly the z_S with $S \ni e$, alive by hypothesis. The movement BFS (R11) from the stack’s hex therefore cannot leave R_e . By hypothesis R_e holds that one stack alone, so no ally blocks the walk, and by (I4) the whole of R_e is within movement range. WALK_AND_ATTACK needs a free hex adjacent to the target; by (I3) the only hexes of R_e adjacent to an enemy are the d_S^e , and d_S^e is adjacent to z_S alone. ■

Lemma D.2 (Alternating triples). *The only pairwise non-adjacent 3-subsets of a hex’s six neighbours are the two alternating triples.*

Proof. Consecutive neighbours on the six-cycle are adjacent to each other, in both row parities; a 3-subset of a 6-cycle with no two consecutive members is an alternating triple. ■ (Machine-checked for both parities, `scripts/test_obstacles.py`.)

So (I1) is a constraint, not a design choice: an enemy that three mutually sealed regions must reach simultaneously is forced to present an alternating triple. This is the one place where hex geometry does real work, and why the construction would not transfer unchanged to a square grid.

D.3 Gadgets

Enemy gadget for $S \in C$, anchored at (X, Y) with Y even: $z_S = (X, Y)$; the three dockings are the alternating triple TOP_LEFT = $(X, Y - 1)$, RIGHT = $(X + 1, Y)$, BOTTOM_LEFT = $(X, Y + 1)$; the other three neighbours are impassable. Each docking is adjacent to z_S and to nothing else in the gadget, so a stack that walks in along one arm cannot slip round to another. (Picture in Section 3.3.)

Element gadget for e : a connected region of free hexes with a two-hex stub at its end carrying p_e .

Corridors connect the dockings to their regions, laid so that no hex of one region is adjacent to a hex of another region or to an enemy hex other than at its own docking.

Lemma D.3 (Adapters). *An orthogonal drawing delivers the three edges of a set-vertex along three of the four axis directions, and which three is not ours to choose; the three*

dockings, by Lemma D.2, are fixed. For each of the four possible triples of incoming directions there is a 9×9 pattern in which three pairwise non-touching paths run from the three used boundary midpoints (“ports”) to the three dockings, each path meeting z_S only at its final hex, with the unused port left impassable.

Proof. Exhibited, one pattern per case, in Appendix C, and machine-checked hex by hex (`scripts/verify_x3c.py::check_enemy_adapters`): each arm is a path, starts at its port, ends at a distinct member of the alternating triple, touches the enemy only at its last hex, no two arms share or touch a hex, and the unused port is untouched. The arms are matched to the dockings in cyclic order, which is what keeps the routing planar inside the box. ■

The box is 9×9 with the enemy at its centre $(4, 4)$ in a local frame whose row 4 is even; the ports are the four boundary midpoints. The parity of the local frame is discharged in step 5 of Lemma D.4.

D.4 The embedding

Lemma D.4 (Embedding). *Let (X, C) be a PLANAR-X3C instance with $N = |X| + |C|$. In time polynomial in N one can compute either*

- *a hex board of polynomial size, a set of impassable hexes, enemy hexes z_S ($S \in C$) and deployment hexes p_e ($e \in X$) satisfying (I1)–(I4), or*
- *a fixed no-instance G_{no} of ARMY-ALLOCATION (a 1×1 board, one slot, one creature of stock one, no enemies, $W = 1$), emitted **only** when the given encoding is malformed or fails one of the polynomial-time no-certificates of step 0— $|X|$ not divisible by 3, $|C| < q$, an element lying in no set, or a failed planarity test.*

“Only when”, not “exactly when”: every input sent to G_{no} is certifiably a no-instance, but most no-instances are *not* sent there—they receive an ordinary board whose game answer is no, which is what a many-one reduction should do. (An “exactly when” would be a decision procedure for PLANAR-X3C.) The second branch is what makes the map a **total** Karp reduction: it outputs a target instance on every source encoding.

Proof, in seven steps. Step 0 disposes of degenerate and malformed inputs; steps 1–2 are graph surgery; step 3 quotes the drawing literature; and steps 4–6 are the hex-level construction, where the adjacency bookkeeping lives.

Step 0: degenerate inputs. Delete repeated members of C (a duplicate 3-set is never needed by an exact cover, so the answer is unchanged); afterwards the incidence graph $G(X, C)$ is simple. If $|X| = 0$, the instance is a **yes**-instance (the empty cover works, $q = 0$) and the algorithm outputs the fixed board built for $X = \{1, 2, 3\}$, $C = \{\{1, 2, 3\}\}$ —a yes-instance of the game—rather than sending the empty encoding through the geometric steps, whose bookkeeping assumes at least one element. If $|X|$ is not divisible by 3, if $|C| < q$, or if some element lies in no member of C , no exact cover exists: output G_{no} . Otherwise every set-vertex of G has degree exactly 3 and every element vertex degree $d_e \geq 1$. Compute a combinatorial planar embedding of G —a rotation system with a choice of outer face—in linear time (Hopcroft–Tarjan, or Boyer–Myrvold, which returns the embedding directly). If G is not planar, the encoding is not a PLANAR-X3C instance and maps to G_{no} , which keeps the reduction total under either convention for defining the promise.

Step 1: degree reduction preserving the rotation. Replace every element vertex e of degree d by a path $v_e^1 - \dots - v_e^d$ of d new vertices, attaching the i -th incident edge of e (in the cyclic order given by the embedding, cut at an arbitrary point) to v_e^i . Call the result G' . *Claim:* G' is plane, $\Delta(G') \leq 3$, and an embedding extending the given one outside a small disc around each e is computable in linear time. Degrees: a set-vertex keeps degree 3; an interior path vertex has two path neighbours and one incidence edge; an endpoint has degree ≤ 2 . Embedding: fix a closed disc Δ_e around e meeting no other vertex and meeting the incident edges in d initial segments; the boundary circle meets them at points $b_1, \dots, b_d$ in the rotation's cyclic order. Cut the circle open between b_d and b_1 and straighten the disc to a rectangle whose top side carries $b_1, \dots, b_d$ left to right; draw the path horizontally across the middle with v_e^i below b_i and join each v_e^i to b_i vertically. The vertical segments are pairwise disjoint and meet the horizontal one only at their own endpoints, so the picture inside the rectangle is plane; outside Δ_e nothing changed. The *cyclic* order of the incidence edges around e becomes the *linear* order along the path, and the cut is made in a face corner, so no crossing is created. Sizes: $|V(G')| = 4|C|$ and $|E(G')| = 6|C| - |X|$, both $O(N)$.

Step 2: connected components. G' need not be connected (the incidence graph of $X = \{1, \dots, 6\}$, $C = \{\{1, 2, 3\}, \{4, 5, 6\}\}$ is a planar yes-instance with two components). Let $G'_1, \dots, G'_t$ be its components, $\sum_i n_i = 4|C|$; run steps 3–6 on each separately, obtaining boards B_i of polynomial size, then **pack**: place them left to right, separated by vertical strips of impassable hexes two hexes wide and surrounded by an impassable border two hexes thick, with each block's vertical offset chosen even (the border allows it). Hexes of distinct blocks then differ by at least 2 in a coordinate, and two hexes at L_∞ distance ≥ 2 are never adjacent (each hex neighbour differs by at most 1 in each coordinate, R1), so no invariant can be violated across blocks and each region stays inside its own block. The packed board remains polynomial. (The implementation packs one level earlier: `planar_embed.py` places the component *drawings* side by side with two empty grid columns between them and scales once, so features of distinct components end at L_∞ distance at least $2\lambda - 2\rho = 32 \geq 2$ by the step-4 separation inequality, and the hexes between them—impassable by default—form exactly the strip this step prescribes. This is the third named subroutine substitution of D.6; the (SEP') check measures every non-incident pair of features, cross-component pairs included, so the substituted packing is verified directly on every built board.)

Step 3: an orthogonal grid drawing. For a connected plane graph H with $\Delta(H) \leq 4$ on n vertices we need a drawing with **(D1)** vertices at distinct points of $\mathbb{Z}^2$; **(D2)** edges as rectilinear grid paths leaving each endpoint along one of the four axis directions, each (vertex, direction) pair used at most once; **(D3)** two edge paths meeting only at a shared endpoint and no path passing through a vertex; **(D4)** all coordinates polynomial in n ; **(D5)** computable in polynomial time. Nothing else—not bend-minimality, not area optimality, not preservation of the step-1 embedding (the correctness argument never refers to it again). Tamassia–Tollis [TT89], in the form of [DG13, Thm. 7.3], supplies exactly this for **connected** 4-plane graphs— $O(n^2)$ area, at most four bends per edge, linear time—and step 2 has already reduced us to the connected case, with $\Delta(G') \leq 3 \leq 4$. The cited theorem bounds the **area** by $O(n^2)$; it does not bound either side by $O(n)$, and we do not need it to. Enclose the drawing Γ in its bounding box and write g for the larger side, so $g = O(n^2)$ in the worst case; every claim below uses only that g is polynomial, so the board of $O(g) \times O(g)$ hexes is polynomial and the unary speed σ is too. Strong NP-hardness is

unaffected. (An earlier draft asserted a $g \times g$ grid with $g = O(n)$, which the cited theorem does not give; the polynomial bound is all the argument uses.)

Step 4: scaling, and the separation inequality. Let $\lambda := 20$ (the scale factor, even) and $\rho := 4$ (the gadget-box radius), and map the drawing into hex coordinates by

$$\Phi(i, j) := (\lambda \cdot (i - i_{\min}) + \omega, \lambda \cdot (j - j_{\min}) + \omega), \quad \omega := 6,$$

where $(i_{\min}, j_{\min})$ is the lower-left corner of Γ 's bounding box; ω is a coordinate offset and has nothing to do with the damage multiplier μ of Section 3.3. **The offset ω must exceed the box radius ρ :** an earlier draft used $\omega = 2$, under which a legal drawing vertex at the bounding-box corner maps to $(2, 2)$ and its 9×9 box reaches $(-2, -2)$, off the board—a genuine construction failure, not a cosmetic one. With $\omega = 6$ every box lies at coordinates ≥ 2 , and ω even keeps every image row $\lambda(j - j_{\min}) + \omega$ even. Fill in each drawn edge: a unit segment of Γ becomes the $\lambda + 1$ hexes on the corresponding axis-aligned run, which is legitimate because the 4-neighbour square grid is a subgraph of hex adjacency in both row parities— (x, y) is adjacent to $(x \pm 1, y)$ and to $(x, y \pm 1)$ in either parity (R1; machine-checked in `test_obstacles.py`).

Two features of Γ that share no point are at L_∞ distance ≥ 1 (their segments lie on integer grid lines with integer endpoints), and Φ scales distances by exactly λ , so disjoint features land at L_∞ distance $\geq \lambda = 20$ (**SEP**). Step 5 replaces a $(2\rho + 1) \times (2\rho + 1)$ box around each vertex image by a gadget and truncates every corridor at the boxes at its ends; every hex of a box is within L_∞ distance ρ of its vertex image, so for two *non-incident* features—two boxes of distinct vertices, a box and a corridor of an edge not incident to it, two corridors of edges sharing no endpoint—

$$L_\infty\text{-dist} \geq \lambda - 2\rho = 20 - 8 = 12 \geq 2, \quad (\text{SEP}')$$

and two hexes at L_∞ distance ≥ 2 are never adjacent. (An earlier draft used $\lambda = 9$: $9 - 8 = 1$, and two boundary cells of unrelated boxes could touch—the repair is this inequality.) Two corridors *incident to the same box* leave it through different ports (by (D2)); outside the box each runs straight along its own axis for at least $\lambda - \rho = 16$ hexes before its first possible bend (bends of Γ sit at grid points, whose images are $\geq \lambda$ away from the vertex image); a point at parameter t on one run and t' on the other are at L_∞ distance $\max(t, t')$ (perpendicular axes) or $t + t'$ (opposite axes), and outside the box $t, t' \geq \rho + 1 = 5$, so in either case the distance is $\geq 5 \geq 2$ and the runs never touch. (Both cases occur: every three of the four axis directions contain exactly one opposite pair.)

Step 5: the gadget boxes. *Set-vertex S .* Its image sits on an even row. Replace the 9×9 box around it by the Lemma D.3 adapter for the triple of axis directions along which Γ delivers the three incident edges; the centre becomes z_S , the arms are assigned to the elements of S by which port each element's corridor arrives at, and every other hex of the box, the unused port included, is impassable. *Parity:* the adapter is stated in a local frame with row 4 even; the translation onto the board shifts rows by $Y - 4$ with Y even, and hex adjacency in offset coordinates depends only on row parity and is invariant under column translation (R1), so the pattern carries over adjacency for adjacency—this is why λ and ω are even.

Element-path vertex v_e^i . Its degree in G' is at most 3, so at least one axis direction is unused. The free hexes of its box are the axis segments from the centre to the used ports plus the centre: a “plus” with at most three arms, connected, contained in the box, meeting

the boundary exactly at the used ports. Adjacencies inside the plus are irrelevant—every hex of it belongs to the single region R_e —which is exactly why the set-vertex needs a hand-built adapter and the element vertex does not.

The deployment stub. Take v_e^1 and one axis direction u unused at it (one exists: $\deg(v_e^1) \leq 2$). Declare free the two hexes at distance 1 and 2 from the centre along u , and set p_e to the one at distance 2. Then p_e is connected to the plus, so it lies in R_e . Nothing else on the board is within distance 12 of the stub (SEP'), so it creates no adjacency anywhere else.

Step 6: closing the board. Declare impassable every hex not made free in steps 4–5; set σ to the hex count of the packed board (legal: creature statistics are input, Section 2.1). The invariants hold: **(I1)** the free neighbours of z_S are exactly the three arm ends—the alternating triple, pairwise non-adjacent by Lemma D.2—and no hex outside the box is adjacent to z_S by (SEP'). **(I2)** the pluses of e 's path vertices, joined by the path-edge corridors, with the incidence corridors, the adapter arms ending at e 's dockings and the stub, form one connected set; distinct regions never touch—their features are non-incident in Γ , so (SEP') applies, and inside an adapter box Lemma D.3 has checked the three arms pairwise non-touching; every free non-enemy hex belongs to exactly one element, so there are exactly $3q$ components. **(I3)** R_e contains p_e ; its hexes adjacent to an enemy are exactly the dockings d_S^e for $S \ni e$ (inside S 's box only the last hex of each arm touches z_S ; outside, (SEP')). **(I4)** σ is the hex count, which is $\geq |R_e|$ for every e . ■

D.5 Correctness

Throughout, fix a feasible allocation $c : X \rightarrow \mathbb{Z}_{\geq 0}$ with $\sum_e c_e \leq 3q$ and an arbitrary play of round 1 against $(\ddagger)$.

Lemma D.5 (No interference). *No player creature dies during round 1, and every player stack gets its action.*

Proof. Under $(\ddagger)$ the defence never initiates. An enemy retaliates only when struck, at most once per round (R12; neither WAIT nor DEFEND consumes the charge, Section 2.4), and its strike deals $\max(1, \lfloor 1 \cdot 1 \cdot 1 \rfloor) = 1$ damage, since $\text{att}(Q) = \text{def}(P) = 1$ gives $\Delta = 0$. A player stack of $c \geq 1$ creatures of 4 hit points absorbs 1 damage without losing a creature (R5, R6). Each player stack strikes at most once, so it receives at most one retaliation. Every living stack takes its terminal action once per round, whichever phase it lands in (R9). ■

Lemma D.6 (Budget). *Let the play kill t enemies. Then $t \leq q$. If $t = q$, then $\sum_e c_e = 3q$; every slot holds exactly one creature; every deployed stack strikes an enemy that dies; and each dead enemy is struck by exactly three distinct one-creature stacks.*

Proof. Each enemy is a single creature of 3 hit points, so it dies exactly when 3 damage has accumulated on it (R8), and that damage comes only from the stacks that struck it. A stack **strikes** at most once per round—WALK_AND_ATTACK ends its turn, and WAIT defers the turn without granting a second strike (R9)—so the striker sets of distinct enemies are disjoint. (The weaker phrasing “a stack acts once per round” would be false under WAIT; the argument runs on strikes throughout.) Let K be the set of dead enemies, $|K| = t$, and A_S the strikers of $S \in K$. By the one-round lemma of Section 2.4 the blow of a stack of c creatures delivers at most $D(c)$, so Lemma 3.2 applied to $(c_a)_{a \in A_S}$ gives $\sum_{a \in A_S} c_a \geq 3$.

Summing over the disjoint A_S :

$$3t \leq \sum_{S \in K} \sum_{a \in A_S} c_a \leq \sum_e c_e \leq 3q, \quad (\dagger)$$

whence $t \leq q$. Suppose $t = q$, so every inequality in $(\dagger)$ is tight. The middle one tight means every allocated creature sits in a stack that struck a member of K —the left sum counts exactly the creatures in striker stacks of K , the right one all allocated creatures. The first one tight, together with $\sum_{a \in A_S} c_a \geq 3$ for each S , forces $\sum_{a \in A_S} c_a = 3$ for every S ; the equality case of Lemma 3.2 then gives $|A_S| = 3$ and $c_a = 1$ for each. Hence the $3q$ allocated creatures sit in exactly $3q$ singleton stacks; a stack occupies one slot and there are exactly $3q = k$ slots, so every slot holds exactly one creature and each struck a member of K . ■

Lemma D.6 uses no geometry whatsoever.

Lemma D.7 (Confinement). *In the situation of Lemma D.6 with $t = q$, when the stack of slot e takes its terminal action, every enemy E_S with $S \ni e$ is still alive. Consequently, by Lemma D.1, that stack can strike only enemies E_S with $S \ni e$.*

Proof. Induction on the realized order of **terminal actions**—the single move, attack, or defend each stack performs. That order is fixed by the phase structure (R9), with a stack that issues **WAIT** taking its terminal action at its later position in the **WAIT** phase; the argument uses only that each stack takes exactly one terminal action, at one position in that order.

Suppose the claim holds for every stack that took its terminal action before slot e . Then every strike so far was delivered from a docking of the striker’s own region: by (I3) the only hexes of R_a adjacent to an enemy are a ’s own dockings, and by the inductive hypothesis every earlier striker was still confined to its region when it moved, because all enemies bounding that region were alive at that moment.

Let E_S be any enemy already dead, and suppose $e \in S$. By Lemma D.6 every stack in play holds one creature and delivers at most $D(1) = 1$ per blow; E_S has 3 hit points and is struck by exactly three stacks over the round, so each blow delivered exactly 1 and E_S died on the third—all three strikers have already acted. Each struck from a hex adjacent to z_S ; by Lemma D.5 no player stack dies, and by R11 a stack that performed **WALK_AND_ATTACK** remains on its approach hex for the rest of the round, so no two strikers can have used the same hex; hence those hexes are the three free neighbours of z_S : the dockings $d_S^{e'}$, one per $e' \in S$. By the previous paragraph the stack that struck from $d_S^{e'}$ is the stack of slot e' . In particular the stack of slot e is among them, so it has already taken its terminal action—contradicting that it is taking it now. Hence no E_S with $S \ni e$ is dead, the boundary of R_e is intact, and Lemma D.1 applies unchanged. ■

Why this lemma is needed at all: a dead unit stops blocking its hex (R4), so the regions are *not* permanently separated—every kill opens a doorway between three of them. Lemma D.7 says the doorway only ever opens for stacks that have already spent their terminal action; without it the no-direction would leak. The induction covers plays in which a stack moves without attacking, defends, or waits—and the three cases are not the same: a stack that moves or defends has *spent* its terminal action, while a stack that waits has *postponed* it, and the induction simply reaches it at its later position. The argument never assumes an acting stack strikes, and the one-round lemma of Section 2.4 covers the only other effect of waiting—a possible damage reduction against a defended target, which only helps the no-direction.

Lemma D.8 (yes $\implies$ yes). *If (X, C) has an exact cover, $G_3(X, C)$ is a yes-instance.*

Proof. Let $C' = \{S_1, \dots, S_q\}$ be an exact cover. Allocate one creature to every slot (total $3q$, exactly the stock). Each e lies in exactly one $S \in C'$; let slot e issue `WALK_AND_ATTACK` to d_S^e against E_S . No stack waits, so every blow lands in the `NORMAL` phase, meets the unraised defence (Section 2.4) and deals its nominal $D(1) = 1$. Every move is legal whatever order the engine imposes: only the three slots of S ever target E_S , so when slot e acts, E_S has absorbed at most 2 and is alive; and d_S^e is free, because in this play every stack performs exactly one `WALK_AND_ATTACK`, ending on a docking of its own region, and R_e holds only slot e 's stack (I2, I3), so doorways opened by earlier kills are never used. Each $S \in C'$ accumulates 3 damage and dies; the destroyed value is $q = W$. ■

Lemma D.9 (yes $\implies$ exact cover). *If $G_3(X, C)$ is a yes-instance, (X, C) has an exact cover.*

Proof. Take an allocation and a play destroying value $\geq q$. Every enemy has value 1, so the number t of dead enemies is $\geq q$; by Lemma D.6, $t \leq q$, hence $t = q$ and the tightness half of Lemma D.6 applies. Let K be the set of dead enemies. Each $S \in K$ is struck by exactly three one-creature stacks, and by Lemma D.7 each striker of E_S is the stack of a slot $e \in S$; the three strikers are distinct and $|S| = 3$, so they are exactly the slots of S . A slot strikes once, so distinct members of K use disjoint slot sets; $|K| = q$ triples use all $3q$ slots. Hence K is a family of q pairwise disjoint 3-sets covering X . ■

Proof of Theorem 3. Lemmas D.8 and D.9 give the equivalence; Lemma D.4 gives a total polynomial-time construction; every number in $G_3(X, C)$ is polynomially bounded, so the reduction is polynomial even under unary encoding, and `PLANAR-X3C` is NP-complete [DF86]—a problem with no numeric parameters, so the hardness obtained through it is strong. Membership is Lemma 2.1. ■

Proof of Corollary 3.1. Fix the allocation $c_e = 1$ for every e , so the instance has no allocation decision and the only choice left is the play. The yes-direction is Lemma D.8, which uses precisely this allocation. For the no-direction, with $c \equiv 1$ every blow delivers at most $D(1) = 1$; an enemy has 3 hit points, so each kill consumes strikes of at least three distinct stacks, and striker sets of distinct dead enemies are disjoint, so killing t enemies consumes at least $3t$ of the $3q$ stacks and $t \leq q$; at $t = q$ every dead enemy is struck by exactly three singleton stacks and every stack strikes a dead enemy—the conclusion of Lemma D.6, obtained here without the resource lemma's equality case (which is only needed to rule out unequal stack sizes). Lemmas D.7 and D.9 then apply verbatim. ■

D.6 Scope of the machine checks

Lemma D.2, Lemma D.3's four adapters, and the square-grid-inside-hex-adjacency fact of step 4 are machine-checked directly. The embedding algorithm of Lemma D.4 is implemented step by step with the lemma's literal constants ($\lambda = 20$, $\rho = 4$, the D.3 adapters unchanged): `embed_lemma.py` is steps 0–6, on top of the planarity and drawing machinery of `planar_embed.py`—with three subroutine substitutions, named here so that a reader who opens the artifact is not surprised. The planarity test is Demoucron–Malgrange–Pertuiset (cubic, chosen because it yields the face set directly), not the linear-time Hopcroft–Tarjan or Boyer–Myrvold that step 0 cites; the drawing is a from-scratch st-ordered visibility construction for maximum degree 3, not [TT89] as quoted in step 3; and step 2's packing of finished boards is realized one level earlier, as a packing of the component drawings (two empty grid columns between components, 2λ hexes after scaling—sound by the

step-4 separation inequality, and measured directly by (SEP'), which checks every non-incident pair of features, cross-component pairs included). All three substitutions are legitimate because the proof consumes only (D1)–(D5), polynomial time and the separation inequality, and `validate_drawing` machine-checks the drawing properties on every build. `verify_embedding.py` feeds every instance family of the published corpora through the algorithm: every board built satisfies (I1)–(I4) and the feature-based (SEP') separation check, every degeneracy skip carries a certificate re-verified against the instance (non-planarity is the DMP test's own verdict, cross-checked by a planted non-planar control), the G_{no} and $|X| = 0$ shortcut branches are exercised— G_{no} is itself played out as a genuine no, and a battery of malformed encodings (wrong arity, non-integers, out-of-range or repeated members, a non-sequence, negative sizes) must each produce a certified G_{no} rather than a crash—and on the smallest boards the full game search runs end-to-end—under the historical and the published constants—and agrees with X3C. The big exhaustive-search suites still take their boards from a compact router (placement by hill-climbing, corridors by BFS with clearance), because the lemma's literal boards are λ times larger per drawing unit than the router's; the router reports failure rather than emitting a board that violates the invariants. What remains a hand proof is the lemma's universal claim—that the algorithm succeeds on *every* planar instance—the scope note of Section 1.2, item 7. The correctness suite additionally verifies, on every built yes-instance, that the unique winning allocation is the all-ones vector predicted by Lemma D.6, and runs a negative control with $\text{def}(Q) = \text{att}(P)$ —destroying Lemma 3.2—under which a no-instance of X3C does turn into a yes-instance of the game, so a pass carries evidence rather than silence.

E Full proofs of Theorems 1, 2 and 4

This appendix makes Theorems 1, 2 and 4, Proposition 1.1 and Corollaries 4.1 and 4.2 self-contained, in the way Appendix D does for Theorem 3: the complete constructions with every creature tuple written out, the geometry, the damage accounting, and both directions of each equivalence. The statements are repeated here so that the appendix can be read on its own; the model is Section 2, the policy (§) and the one-round lemma are Section 2.4, and rule tags R1–R17 resolve to engine lines in Appendix B.

Three of the repairs below change what the body says rather than only how it says it, and they are flagged where they occur: Theorem 1's no-direction is re-derived over disjoint striker sets and uses no reach hypothesis at all (Section E.2); Proposition 1.1's matching hypothesis is quantified over *every* position reachable in round-1 play, not only over the starting position, because a dead unit stops blocking its hex (Section E.3); and all three reductions are made total (Section E.1).

E.1 Conventions common to the three constructions

The grid. We use the offset (“even-row shifted”) coordinates of R1 throughout, in the concrete form printed in Appendix C: writing (x, y) for column x of row y , the six neighbours of (x, y) are

$$(x \pm 1, y), \quad (x - \epsilon, y - 1), (x + 1 - \epsilon, y - 1), \quad (x - \epsilon, y + 1), (x + 1 - \epsilon, y + 1), \quad \epsilon := y \bmod 2.$$

So from an **even** row the two upper neighbours sit at columns x and $x + 1$, and from an **odd** row at columns $x - 1$ and x . Distance is the axial formula of R2: with $A(x, y) := x + \lfloor y/2 \rfloor$,

$\delta A := A(q) - A(p)$ and $\delta y := y_q - y_p$,

$$\text{dist}(p, q) = \begin{cases} \max(|\delta A|, |\delta y|) & \text{if } \delta A \text{ and } \delta y \text{ are both } \geq 0 \text{ or both } < 0, \\ |\delta A| + |\delta y| & \text{otherwise.} \end{cases} \quad (\text{R2})$$

Every coordinate computation below is done in (A, y) .

($\star$) **in full.** ($\star$) is the specialization announced in Section 3, in its complete form: fix $\alpha := 1$ and give *every player type* $\text{att} = \text{def} = 1$, *every enemy type* $\text{att} = \text{def} = 1$, and every enemy type flat damage $\text{dmg}_{\min} = \text{dmg}_{\max} = 1$. Then $\Delta = 0$ on a player $\rightarrow$ enemy blow *and* on the retaliation, so $f_{\text{att}} = f_{\text{def}} = 1$ in both directions (R7) and a stack of $c \geq 1$ creatures of flat per-creature damage $d \geq 1$ delivers nominal damage

$$\text{dmg} = \max(1, \lfloor cd \rfloor) = cd. \quad (\star)$$

The lower bound $d \geq 1$ is not decoration: it is the model’s own domain restriction $1 \leq \text{dmg}_{\min} \leq \text{dmg}_{\max}$ (Section 2.2), and it is what makes the clamp $\max(1, \cdot)$ inert here. Every claim below that says “nominal is dealt” uses $c \geq 1$ and $d \geq 1$; every claim that quantifies over an arbitrary play says “at most nominal”, as Section 2.4 requires.

An earlier form of ($\star$) fixed only the player’s attack and the enemy’s defence, which left three components of the constructed tuples undetermined—the player’s defence and the enemies’ attack and damage—so that the reductions did not output a single well-defined ARMY-ALLOCATION instance. The form above fixes them, and Lemma E.2 is where they are used.

Complete tuples. A creature type is the tuple $(\text{att}, \text{def}, \text{dmg}_{\min}, \text{dmg}_{\max}, \text{hp}, \text{spd}, \text{flags})$ together with its *value* (Section 2.2). In all three constructions $\text{flags} = \emptyset$: every creature is melee, single-hex, non-shooting, with the default single retaliation charge—the fragment H3-det-melee of Section 2.2. Every player type has *value* = 0, so the objective reads enemy values only, as the Problem of Section 2.3 prescribes. With ($\star$) fixing four of the numbers and *flags* and *value* fixed here, each construction below has only d , hp , spd and the stock left to name, and it names all four.

Totality. A Karp reduction must output a target instance on every source encoding, and the constructions below presuppose a nonempty, well-formed source instance. All three are made total by the same three-way branch, in the pattern of Lemma D.4:

1. If the encoding is malformed, or fails one of the syntactic checks listed with the construction—each of which certifies in polynomial time that no solution exists—output the fixed no-instance G_{no} of Lemma D.4: a 1×1 board, one slot, one creature of stock one, no enemies, $W = 1$. Its destroyed value is $0 < 1$, so it is a no-instance, and $W = 1 \in \mathbb{Z}_{>0}$ as the Problem requires.
2. If the source instance is *empty*—no numbers at all—it is a **yes**-instance (the empty selection works), so output instead the instance the construction builds from a fixed nonempty source yes-instance, named with each construction. Sending the empty encoding through the geometric steps is what must be avoided: it yields an empty board and $W = 0 \notin \mathbb{Z}_{>0}$.
3. Otherwise, run the construction.

“Only when”, not “exactly when”, again: every encoding routed to G_{no} is certifiably a no, and most no-instances are not routed there but receive an ordinary board whose game answer is no.

Lemma E.1 (one blow, and where it is issued). *Fix any allocation and any play of round 1 against $(\ddagger)$ in any of the three constructions. Then every player stack delivers **at most one blow** in the round, and if it delivers one, it is issued at the stack’s terminal action, from the stack’s own deployment hex.*

Proof. A stack has exactly one terminal action per round—move, attack, or defend—and WAIT is not one: it postpones the terminal action into the WAIT phase without moving or striking (R9, Section 2.4). The only player action that strikes is WALK_AND_ATTACK, which moves and strikes as a single terminal action (R11), so at the moment it is issued the stack still stands where it stood at the start of the round, namely on its deployment hex. The only other blow a player stack could deliver is a retaliation, and under $(\ddagger)$ the defence never initiates an attack (Section 2.4), so no player stack ever retaliates. Every type used below has $\text{flags} = \emptyset$, so no ability grants a second strike. ■

Lemma E.1 is the step that an earlier draft of Section 3.4 compressed into “each type has stock one, so each stack strikes at most one enemy”. Stock one gives that each *type* occupies at most one slot; what gives one blow per stack is one terminal action per round together with the absence of enemy initiative. The distinction is exactly the one whose neglect produced the first, wrong version of Theorem 1 (Section 2.4).

Lemma E.2 (retaliation is inert). *In round 1 of any of the three constructions, under $(\star)$ and $(\ddagger)$: a player stack takes at most one retaliation blow; that blow lands strictly after the stack’s own blow; it deals exactly 1 damage; and no player creature dies.*

Proof. Under $(\ddagger)$ an enemy never initiates, so the only damage a player stack can take is retaliation, which by R12 resolves *after* the attacker’s blow has been applied. A stack strikes at most once (Lemma E.1), so it draws at most one retaliation. Under $(\star)$ the retaliating enemy is a single creature of flat damage 1 with $\Delta = \text{att}(\text{enemy}) - \text{def}(\text{player}) = 1 - 1 = 0$, so its blow deals $\max(1, \lfloor 1 \cdot 1 \cdot 1 \cdot 1 \rfloor) = 1$ (R7). Every player type below has $hp = 5$, so a stack of $c \geq 1$ creatures absorbs 1 point in its pool (*firstHPleft* drops from 5 to 4) and its count is unchanged (R5, R6). ■

Two consequences are used without further comment. No blow already delivered is affected by a retaliation, so the damage bookkeeping of each construction may ignore retaliation entirely; and the objective reads the values of enemy types only (Section 2.3), so player losses—of which there are none—could not affect it in any case.

Lemma E.3 (reach is bounded by the hex metric). *Let a player stack of speed s stand on hex p in some position of the battle, and suppose it strikes an enemy occupying hex z . Then $\text{dist}(p, z) \leq s + 1$, where dist is the R2 distance. The bound holds **whatever hexes are free at the time**.*

Proof. WALK_AND_ATTACK walks a path of enterable hexes of length at most s from p to a hex h adjacent to z , and strikes from h (R10, R11). It suffices that the R2 distance is the graph distance of the R1 adjacency, since a BFS distance over any subgraph is at least the graph distance over the whole grid; then $\text{dist}(p, h) \leq s$ and $\text{dist}(p, z) \leq s + 1$.

In the coordinates $(A, y) = (x + \lfloor y/2 \rfloor, y)$ the six R1 steps become, in *both* row parities, exactly

$$\pm(1, 0), \quad \pm(0, 1), \quad \pm(1, 1)$$

(the two same-row steps give $\pm(1, 0)$; the step to column $x + 1 - \epsilon$ of row $y - 1$ and the step to column $x - \epsilon$ of row $y + 1$ give $(0, -1)$ and $(0, +1)$; the remaining two give $-(1, 1)$ and $+(1, 1)$). Let H be the hexagon with these six vectors as its vertices. The R2 formula is the Minkowski gauge of H : on the two quadrants where δA and δy agree in sign, the boundary of H is the polyline $\max(|\delta A|, |\delta y|) = 1$, and on the other two it is $|\delta A| + |\delta y| = 1$. A gauge is subadditive and takes the value 1 on each of the six steps, so a walk of length L realizes a displacement of gauge at most L . Conversely, when δA and δy agree in sign, $\min(|\delta A|, |\delta y|)$ diagonal steps with vertical or same-row steps for the remainder realize the displacement in $\max(|\delta A|, |\delta y|) = \text{dist}$ steps; when they differ in sign no diagonal helps, and $|\delta A|$ same-row with $|\delta y|$ vertical steps realize it in $|\delta A| + |\delta y| = \text{dist}$ steps. ■

Lemma E.3 is the tool that disposes of the dynamic-reach objection, and it is worth saying why it is stated in this reach-independent form. A dead unit stops blocking its hex (R4), so every kill can open a doorway, and a reach argument that reads off the *starting* free graph proves nothing about later positions—this is the phenomenon Theorem 3 spends Lemma D.7 on. In Theorems 1, 2 and 4 no such lemma is needed, because the separation is metric: the bound of Lemma E.3 does not look at which hexes are free, so freeing hexes cannot violate it.

E.2 Theorem 1

Theorem 1. ARMY-ALLOCATION is NP-complete, already for instances with $R = 1$, a *single creature type* in the player's army, one creature per enemy stack, no obstacles, and a battlefield of one row.

Source problem. PARTITION ([GJ79, SP12], [Kar72]): given positive integers $a_1, \dots, a_n$ with $\sum_i a_i = 2B$, is there $S \subseteq [n]$ with $\sum_{i \in S} a_i = B$?

The instance $G(a)$. Given $a = (a_1, \dots, a_n)$ with $n \geq 1$, every $a_i \in \mathbb{Z}_{>0}$ and $\sum_i a_i = 2B$:

- **Battlefield.** One row of $5n$ hexes, indexed $0, \dots, 5n - 1$; no obstacles. Block j ($1 \leq j \leq n$) occupies hexes $5(j - 1), \dots, 5j - 1$, and within it

$$p_j := 5(j - 1) \quad (\text{deployment hex of slot } j), \quad e_j := 5(j - 1) + 1 \quad (\text{hex of } E_j),$$

the remaining three hexes of the block being empty. There are $k = n$ slots.

- **Player army.** One creature type C with $\text{att} = \text{def} = 1$ (that is $(\star)$), flat damage $d = 1$, $\text{hp} = 5$, $\text{spd} = 2$, $\text{flags} = \emptyset$, $\text{value} = 0$, and stock exactly B .
- **Defence.** For each j , the stack E_j is **one** creature of a type with $\text{att} = \text{def} = 1$, flat damage 1, $\text{hp} = a_j$, $\text{spd} = 1$, $\text{flags} = \emptyset$, $\text{value} = a_j$, at e_j , playing $(\dagger)$.
- **Question.** $R = 1$, $W = B$.

Since every $a_i \geq 1$ and $n \geq 1$ we have $B \geq 1$, so the stock and $W = B \in \mathbb{Z}_{>0}$ are legal. The board is listed hex by hex in $5n$ cells and the numbers a_j occur only as hit points and values, in binary, so $G(a)$ is computable in time polynomial in the binary encoding of a . (A map polynomial in the binary length is a fortiori polynomial in the unary one; Theorem 1 is *weak* hardness not because the map fails in unary but because Proposition 1.1's pseudo-polynomial algorithm decides the unary problem.)

Totality. The three-way branch of Section E.1, instantiated: route to G_{no} every encoding that is malformed, or has some $a_i \leq 0$, or has $\sum_i a_i$ odd—in the last case no S can sum to the non-integer $\sum_i a_i/2$, so a no-certificate is at hand; route the empty encoding $n = 0$, which is a PARTITION yes-instance because the empty subset sums to $0 = B$, to the fixed instance $G((1, 1))$, a yes-instance of ARMY-ALLOCATION by Lemma E.6; and otherwise output $G(a)$.

Lemma E.4 (separation in the corridor). *In $G(a)$, $\text{dist}(p_j, e_j) = 1$ for every j , and $\text{dist}(p_j, e_{j'}) = |5(j - j') - 1| \geq 4$ for $j \neq j'$. A player stack has strike radius $\text{spd} + 1 = 3$ (R11), so in every position of round 1 a stack standing on p_j can strike E_j and no other enemy.*

Proof. On a one-row board R2 degenerates to $|\delta x|$, so $\text{dist}(p_j, e_{j'}) = |5(j-1) - 5(j'-1) - 1| = |5(j - j') - 1|$, which is 1 at $j' = j$ and, for $|j - j'| \geq 1$, at least $|5 - 1| = 4$. By Lemma E.3 a stack of speed 2 on p_j can strike only enemies at distance at most 3, and E_j is the only one. ■

The two numbers here are the reduction’s one tight margin, and the phrase “consecutive blocks are 4 apart”, which an earlier body draft used, named neither of them: consecutive blocks are **5** apart (block j starts at $5(j - 1)$ and block $j + 1$ at $5j$); what equals 4 is $\min_{j' \neq j} \text{dist}(p_j, e_{j'})$, attained at $j' = j - 1$, the forward gap being 6. Block width 5 is minimal: at width 4 the backward gap would be $|4 - 1| = 3$, exactly the strike radius, and slot j would reach E_{j-1} .

Lemma E.5 (the threshold at E_j). *Fix an allocation $(c_1, \dots, c_n)$ with $\sum_j c_j \leq B$ and any play of round 1. Let A_j be the set of slots whose stack struck E_j during the round. Then*

1. $A_j \subseteq \{j\}$, and the sets $A_1, \dots, A_n$ are pairwise disjoint;
2. the total damage E_j absorbs is at most $\sum_{i \in A_j} c_i$, so E_j dead implies $\sum_{i \in A_j} c_i \geq a_j$;
3. if $c_j \geq a_j$ and the stack of slot j issues **WALK_AND_ATTACK** against E_j without waiting, E_j dies.

Proof. (1) By Lemma E.1 every blow a player stack delivers is issued at its terminal action from its deployment hex, so by Lemma E.4 the stack of slot i can only strike E_i ; hence $A_j \subseteq \{j\}$ and distinct A_j are disjoint. (Disjointness in fact needs no geometry: a stack delivers at most one blow, so it belongs to at most one A_j .)

(2) By the one-round lemma of Section 2.4 every blow delivers at most its nominal damage, and under $(\star)$ the nominal damage of the stack of slot i is $c_i \cdot d = c_i$ ($d = 1$). Damage accumulates in E_j ’s pool and excess is discarded (R8), so the absorbed total is at most $\sum_{i \in A_j} c_i$. The stack E_j is a single creature, so by R5 its pool is $(\text{fullUnits}, \text{firstHPleft}) = (0, a_j)$ and count = 1 (R6); by R8, $\text{kills}(D) = 0$ for $D < a_j$ and $\text{kills}(D) = \min(1 + \lfloor (D - a_j)/a_j \rfloor, 1) = 1$ otherwise. So E_j dies exactly when the damage it absorbs reaches a_j , and death forces $\sum_{i \in A_j} c_i \geq a_j$.

(3) A non-waiting blow in round 1 lands in the **NORMAL** phase, strictly before every enemy’s postponed **DEFEND**, so it meets the un-raised defence (Section 2.4) and delivers its full nominal $c_j \geq a_j$ (here $c_j \geq a_j \geq 1$ and $d = 1$, so $(\star)$ applies). By Lemma E.4 the approach hex may be p_j itself, at the approach reached by an empty walk, p_j being adjacent to E_j by Lemma E.4, so the strike is legal whatever else is on the board; and by Lemma E.2 no player creature dies, so the stack is alive to take its action (R9). ■

Lemma E.6 (PARTITION yes $\implies$ game yes). *If a is a PARTITION yes-instance then $G(a)$ is a yes-instance.*

Proof. Let $S \subseteq [n]$ with $\sum_{j \in S} a_j = B$. Allocate $c_j := a_j$ for $j \in S$ and $c_j := 0$ otherwise; the total is B , exactly the stock, so the allocation is feasible, and each slot receives at most one type because there is only one. Let every nonempty stack issue `WALK_AND_ATTACK` against the enemy of its own block, from its own hex, without waiting. Each such E_j is struck only by slot j , so it stands at full pool when struck; by Lemma E.5(3) it dies. The destroyed value is $\sum_{j \in S} a_j = B = W$. ■

Lemma E.7 (game yes $\implies$ PARTITION yes). *If $G(a)$ is a yes-instance then a is a PARTITION yes-instance.*

Proof. Fix a feasible allocation $(c_1, \dots, c_n)$, $\sum_j c_j \leq B$, and a play destroying value at least $W = B$. The only creatures carrying value are the E_j , of value a_j each, so with $S := \{j : E_j \text{ dead at the end of round 1}\}$ the destroyed value is exactly $\sum_{j \in S} a_j \geq B$. By Lemma E.5(2), $\sum_{i \in A_j} c_i \geq a_j$ for every $j \in S$, and by Lemma E.5(1) the A_j are pairwise disjoint subsets of $[n]$. Hence

$$B \leq \sum_{j \in S} a_j \leq \sum_{j \in S} \sum_{i \in A_j} c_i \leq \sum_{i=1}^n c_i \leq B, \quad (\dagger_1)$$

so every inequality is an equality; in particular $\sum_{j \in S} a_j = B$ and S solves PARTITION. ■

Two remarks on $(\dagger_1)$. First, the chain uses *no reach hypothesis*: only that the striker sets are disjoint (Lemma E.1) and that each dead E_j absorbed at least a_j from them. Lemma E.4 is therefore needed for the yes-direction (where slot j must be adjacent to E_j) but not for the no-direction. Consequently an earlier version of Section 2.1, which warned that widening the player’s speed would break Theorem 1, was too strong: translation and speed-widening both preserve the yes-direction, and the no-direction never looked at the board. What a wider speed breaks is Proposition 1.1’s hypothesis, which is a different statement about a different family—and that is what Section 2.1 now says. Second, the chain bounds the *total* damage E_j absorbs, not the size of one blow; an earlier body draft’s “its blow delivers at most c_j ” bounded the wrong quantity even though the number is the same.

Proof of Theorem 1. Membership in NP is Lemma 2.1. The map $a \mapsto G(a)$, extended by the totality branch above, is computable in time polynomial in the binary encoding of a and, by Lemmas E.6 and E.7 together with the two fixed instances of the branch, sends yes-instances to yes-instances and no-instances to no-instances. The constructed instances have $R = 1$, a single player creature type, one creature per enemy stack, no obstacles and one row. PARTITION is NP-complete [Kar72], so ARMY-ALLOCATION is NP-complete on this family. ■

E.3 Proposition 1.1, and what “matching reach” has to mean

The family Theorem 1 constructs admits a pseudo-polynomial algorithm, which is what makes the weak hardness of Theorem 1 tight. Stating the algorithm’s hypothesis correctly takes one definition, because the naive reading—“at the start, slot j reaches exactly one enemy”—is a statement about the starting position, while the algorithm needs a statement about the whole round.

Definition E.8 (persistent matching reach). An instance with $R = 1$, defence $(\ddagger)$ and k slots has **persistent matching reach** if there is a bijection $j \mapsto E_j$ from the slots onto the enemy stacks such that for every feasible allocation, in every position of round

1 reachable from the starting position that allocation induces by legal player actions and the defence's ($\dagger$) actions, for every slot j whose stack has *not yet taken its terminal action*, the set of enemies that stack can strike is exactly $\{E_j\}$.

A stack that has not taken its terminal action has not moved and has not struck (Lemma E.1), so it still stands on p_j ; “the set of enemies it can strike” is therefore the set of enemies E for which some free hex adjacent to E 's hex is within movement range of p_j over the free hexes of the current position. The quantification over positions is the whole point of the definition:

Remark. A dead unit stops blocking its hex (R4), so the free graph grows during the round and reach is *dynamic*. A hypothesis imposed only on the starting position would leave open that a kill opens a doorway through which a second slot reaches a third slot's enemy—exactly the leak that Theorem 3 closes with Lemma D.7. Definition E.8 closes it by fiat and Lemma E.9 discharges it, for the family at hand, with a criterion that never reads the free graph.

Lemma E.9 (a metric test for Definition E.8). *Suppose the enemies are $E_1, \dots, E_k$ at hexes $e_1, \dots, e_k$, the slots have deployment hexes $p_1, \dots, p_k$ and speeds $s_1, \dots, s_k$, and*

1. p_j is adjacent to e_j for every j , and
2. $\text{dist}(p_j, e_{j'}) > s_j + 1$ for all $j \neq j'$.

Then the instance has persistent matching reach, with the bijection $j \mapsto E_j$.

Proof. Fix a reachable position and a slot j whose stack has not taken its terminal action; it stands on p_j .

Containment. If that stack strikes an enemy at hex z then $\text{dist}(p_j, z) \leq s_j + 1$ by Lemma E.3, and hypothesis (2) rules out every $e_{j'}$ with $j' \neq j$. Note that Lemma E.3 is insensitive to which hexes are free, so this holds in every reachable position, however many hexes earlier kills have freed.

Non-emptiness. By containment applied to every slot, no stack other than j 's can ever strike E_j ; and j 's stack has not struck. So E_j is alive. Its hex is adjacent to p_j by (1), so WALK_AND_ATTACK against E_j with the empty walk and approach hex p_j is legal (R11), and E_j is strikable. ■

Lemma E.10 (the family of Theorem 1 qualifies). *Every instance $G(a)$ built in Section E.2 has persistent matching reach.*

Proof. Lemma E.4 gives $\text{dist}(p_j, e_j) = 1$ and $\text{dist}(p_j, e_{j'}) \geq 4 > 3 = \text{spd} + 1$ for $j' \neq j$; apply Lemma E.9. ■

Proposition 1.1. *On the family of Theorem 1—a single creature type of flat damage $d \geq 1$ and stock B , $R = 1$, policy ($\dagger$), damage under ($\star$), **one creature per enemy stack**, and **persistent matching reach** in the sense of Definition E.8—ARMY-ALLOCATION is solvable in $O(kB)$ time and $O(B)$ space.*

Proof. Write t_j for the hit points and v_j for the value of the single creature of E_j , and $b_j := \lceil t_j/d \rceil$; b_j is well defined because $d \geq 1$ (Section E.1). We show that the optimum destroyed value equals

$$\text{OPT} = \max \left\{ \sum_{j \in S} v_j : S \subseteq [k], \sum_{j \in S} b_j \leq B \right\}, \quad (\text{K})$$

a 0-1 knapsack over k items, which the textbook dynamic program over (slot prefix, consumed budget) solves in $O(kB)$ time and $O(B)$ space by keeping one budget row.

The optimum is at most (K). Fix a feasible allocation $(c_1, \dots, c_k)$, $\sum_j c_j \leq B$, and any play of round 1; let S be the set of j with E_j dead. By Lemma E.1 every blow is issued at its stack's terminal action from its deployment hex, so Definition E.8 applies to it: the stack of slot i strikes only E_i , hence E_j is struck only by slot j . By the one-round lemma every blow delivers *at most* its nominal damage, which is $c_j d$ under $(\star)$, and damage accumulates in the pool with overkill discarded (R8). Since E_j is a single creature of t_j hit points, R5, R6 and R8 give $\text{count} = 1$, $\text{firstHPleft} = t_j$ and $\text{kills}(D) = 1$ exactly when $D \geq t_j$. So $j \in S$ forces $c_j d \geq t_j$, i.e. $c_j \geq b_j$. Therefore $\sum_{j \in S} b_j \leq \sum_{j \in S} c_j \leq B$ and the destroyed value $\sum_{j \in S} v_j$ is one of the sums in (K).

The optimum is at least (K). Let S attain (K). Allocate $c_j := b_j$ for $j \in S$ and $c_j := 0$ otherwise, feasible since $\sum_{j \in S} b_j \leq B$. Instruct every nonempty stack to issue WALK_AND_ATTACK against its own E_j , without waiting; no stack waits, so every blow lands in the NORMAL phase and meets the un-raised defence (Section 2.4) and delivers its full nominal $c_j d \geq t_j$ (here $c_j = b_j \geq 1$ and $d \geq 1$). Each such action is legal when it is taken: the stack has not yet taken its terminal action, so by Definition E.8 the set of enemies it can strike is exactly $\{E_j\}$ —in particular E_j is alive and an approach hex is free, *in that position*, whatever the earlier movers have done. This is where the quantification over positions earns its keep: no separate argument is needed that an earlier mover cannot block the route, because any position in which it did would be a reachable position violating Definition E.8. Every E_j , $j \in S$, therefore dies and the destroyed value is $\sum_{j \in S} v_j$. ■

Both extra hypotheses are load-bearing. *One creature per enemy stack:* against a stack of several creatures a non-finishing blow still kills whole creatures and still scores, the per-slot value is a staircase in c_j rather than a threshold, and (K) is false—round 10 exhibited a matching-reach instance with six-creature stacks on which the threshold rule returns 0 against a true optimum of 3 (open problem 4, Section 7). *Persistent matching reach:* it does not follow from the other restrictions of Theorem 1's statement, not even on one row. Take hexes 0, ..., 7; one player type of flat damage 1, $hp = 5$, $spd = 1$, stock $B = 2$; slot 1 at hex 1 and slot 2 at hex 7; single-creature enemies E_1 at hex 0 and E_2 at hex 2, each of 1 hit point and value 1; $R = 1$, $W = 2$. Every restriction in Theorem 1's statement holds. Slot 1 is adjacent to both enemies, so the reach is not a matching; slot 2 has strike radius 2 and reaches nothing. The true optimum is 1—slot 1 strikes once—while (K) with $b_1 = b_2 = 1$ and $B = 2$ returns 2, and the dynamic program, which indexes its items by slots through the enemy each slot uniquely reaches, is not even well defined. What Proposition 1.1 shows tight is the family the reduction *constructs*, not the family Theorem 1 quantifies over.

E.4 Theorem 2

Theorem 2. ARMY-ALLOCATION is **strongly** NP-hard, already for $R = 1$, no obstacles, no abilities, one creature per enemy stack, and instances in which every player creature type has stock exactly one.

Source problem. 3-PARTITION ([GJ79, SP15], strongly NP-complete): given $3m$ positive integers $a_1, \dots, a_{3m}$ and a bound T with $\sum_i a_i = mT$ and $T/4 < a_i < T/2$ for all i , is there a partition of $[3m]$ into m triples each summing to T ? It remains NP-complete with all a_i bounded by a polynomial in m , i.e. written in unary.

The instance $G_3(a, T)$. Given such an instance with $m \geq 1$:

- **Battlefield.** Three rows ($y \in \{0, 1, 2\}$) and $8m + 2$ columns; no obstacles. For $g = 1, \dots, m$ put $X_g := 8(g - 1) + 1$ and place

$$e_g := (X_g, 1), \quad q_g^1 := (X_g - 1, 1), \quad q_g^2 := (X_g, 0), \quad q_g^3 := (X_g, 2).$$

There are $k = 3m$ slots, with deployment hexes $q_1^1, q_1^2, q_1^3, \dots, q_m^1, q_m^2, q_m^3$.

- **Player army.** $3m$ creature types $C_1, \dots, C_{3m}$, with $\text{att}(C_i) = \text{def}(C_i) = 1$ (that is $(\star)$), flat damage $\text{dmg}_{\min} = \text{dmg}_{\max} = a_i$, $\text{hp} = 5$, $\text{spd} = 2$, $\text{flags} = \emptyset$, $\text{value} = 0$, and **stock exactly one**.
- **Defence.** E_g is **one** creature with $\text{att} = \text{def} = 1$, flat damage 1, $\text{hp} = T$, $\text{spd} = 1$, $\text{flags} = \emptyset$, $\text{value} = 1$, at e_g , playing $(\dagger)$.
- **Question.** $R = 1$, $W = m$.

Every number is bounded by a polynomial in m once the a_i are, and the board has $3(8m + 2) = O(m)$ hexes, so the construction is polynomial even under **unary** encoding of the a_i —which is what makes the hardness strong. The board is wide enough: q_1^1 sits in column 0 and e_m in column $X_m = 8m - 7 \leq 8m + 1$.

Totality. As in Section E.1: route to G_{no} every encoding that is malformed, has a number of items not divisible by 3, has some $a_i \leq 0$, or fails $\sum_i a_i = mT$ or $T/4 < a_i < T/2$ —each check is a polynomial-time no-certificate under the convention that an encoding outside the problem’s domain is a no; route the empty encoding $m = 0$, a yes-instance, to the fixed instance $G_3((1, 1, 1), 3)$, a yes-instance of ARMY-ALLOCATION by Lemma E.13; otherwise output $G_3(a, T)$.

Lemma E.11 (three seats, and separation at distance 7). *In $G_3(a, T)$ each of q_g^1, q_g^2, q_g^3 is a neighbour of e_g , the three are distinct, and for $g' \neq g$*

$$\text{dist}(q_g^r, e_{g'}) \geq 7 \quad (r = 1, 2, 3),$$

with equality attained, e.g. at $q_{g+1}^1 = (X_g + 7, 1)$ against e_g . Since a player stack has strike radius $\text{spd} + 1 = 3$ (R11), in every position of round 1 a stack standing on q_g^r can strike E_g and no other enemy.

Proof. Row 1 is odd, so by R1 the six neighbours of $(X_g, 1)$ are $(X_g \pm 1, 1)$, $(X_g - 1, 0)$, $(X_g, 0)$, $(X_g - 1, 2)$, $(X_g, 2)$; the three hexes q_g^1, q_g^2, q_g^3 are among them and are pairwise distinct.

For the separation, pass to the axial coordinates $A(x, y) = x + \lfloor y/2 \rfloor$ of R2, in which the group- g hexes read

$$\begin{aligned} A(e_g) &= X_g \quad (y = 1), & A(q_g^1) &= X_g - 1 \quad (y = 1), \\ A(q_g^2) &= X_g \quad (y = 0), & A(q_g^3) &= X_g + 1 \quad (y = 2), \end{aligned}$$

and put $D := X_{g'} - X_g = 8(g' - g)$, so $|D| \geq 8$. Then $(\delta A, \delta y)$ from q_g^r to $e_{g'}$ is $(D + 1, 0)$, $(D, 1)$, $(D - 1, -1)$ for $r = 1, 2, 3$, and R2 gives

	$(\delta A, \delta y)$	$D \geq 8$	$D \leq -8$
$r = 1$	$(D + 1, 0)$	$\max(D + 1, 0) = D + 1 \geq 9$	$ D + 1 + 0 = D - 1 \geq 7$
$r = 2$	$(D, 1)$	$\max(D, 1) = D \geq 8$	$ D + 1 \geq 9$
$r = 3$	$(D - 1, -1)$	$(D - 1) + 1 = D \geq 8$	$\max(D - 1 , 1) = D + 1 \geq 9$

(the mixed-sign rows use the $|\delta A| + |\delta y|$ branch, the others the max branch). The minimum is 7, attained at $r = 1$, $D = -8$, i.e. at q_{g+1}^1 against e_g . Since $7 > 3$, Lemma E.3 leaves E_g as the only enemy a stack on q_g^r can ever strike. ■

An earlier body draft said “at distance at least 6”—true but slack, and its derivation subtracted a unit twice; the computation above is in the paper’s own metric and is tight. The bound is machine-checked on every constructed instance in the worst case for the player, with all $3m$ slots occupied (`brute_force.py`, `check_geometry_3partition`).

Lemma E.12 (damage accounting). *Fix an allocation and any play of round 1 of $G_3(a, T)$. For $g \in [m]$ let $S_g \subseteq [3m]$ be the set of types whose stack struck E_g . Then the S_g are pairwise disjoint, $|S_g| \leq 3$, the damage E_g absorbs is at most $\sum_{i \in S_g} a_i$, and E_g is dead at the end of the round only if $\sum_{i \in S_g} a_i \geq T$.*

Proof. A slot is homogeneous and every type has stock one, so a slot holds at most one creature and a type occupies at most one slot. By Lemma E.1 each stack delivers at most one blow, from its own deployment hex, and by Lemma E.11 that blow can only reach the enemy of its own group; hence each type lies in at most one S_g (disjointness) and only the three slots of group g feed S_g , so $|S_g| \leq 3$. A stack of type C_i holds one creature, so under $(\star)$ its nominal blow is $1 \cdot a_i = a_i$, and by the one-round lemma of Section 2.4 it delivers at most that. (At most, not strictly less: the formula clamps at 1 and $a_i = 1$ is legal, and only the inequality is used.) No further damage reaches E_g : under $(\ddagger)$ no enemy initiates, so no player stack ever delivers a retaliation (Lemma E.1). Damage accumulates in E_g ’s pool with overkill discarded (R8), and E_g is a single creature, so R5 and R6 give $(\text{fullUnits}, \text{firstHPleft}) = (0, T)$ and $\text{count} = 1$, whence by R8 $\text{kills}(D) = 0$ for $D < T$ and $\text{kills}(D) = \min(1 + \lfloor (D - T)/T \rfloor, 1) = 1$ for $D \geq T$: death requires the absorbed total, hence $\sum_{i \in S_g} a_i$, to reach T . ■

Three independent facts sit inside that proof and each is used again below: a one-creature stack of C_i delivers at most a_i ; each stack strikes at most once; and *no player stack ever delivers retaliation damage*. The third is the one whose absence produced the paper’s recorded first error (Section 4.3).

Lemma E.13 (3-PARTITION yes $\implies$ game yes). *If (a, T) is a 3-PARTITION yes-instance then $G_3(a, T)$ is a yes-instance.*

Proof. Let $\{G_1, \dots, G_m\}$ be a partition of $[3m]$ into triples with $\sum_{i \in G_g} a_i = T$. Allocate the three types of G_g to the three slots q_g^1, q_g^2, q_g^3 , one creature each—feasible, since each type has stock one and each slot receives one type. Instruct every stack to issue WALK_AND_ATTACK against the enemy of its own group, from its own hex, without waiting. We check that this play is legal and that it kills every E_g ; each item is a step the body’s sketch left implicit.

Every stack acts. Under $(\ddagger)$ no enemy initiates, and by Lemma E.2 the only damage a player stack takes is one retaliation of 1 point against $hp = 5$, delivered strictly after that stack’s own blow. So no player stack is killed or weakened before acting, and every living stack takes its terminal action once in the round (R9).

Every blow is legal. q_g^r is adjacent to e_g (Lemma E.11), so the approach hex is the stack's own hex and the walk is empty (R11); no ally can block it and no earlier mover can occupy it, since no stack in this play ever leaves its deployment hex.

Every blow delivers exactly a_i . No player stack waits, so all $3m$ player blows land in the NORMAL phase, while under (‡) each E_g spends its NORMAL activation on WAIT and issues its terminal DEFEND only in the later WAIT phase (R9, Section 2.4). Every blow therefore meets the un-raised defence $\text{def}(E_g) = 1$, so $\Delta = 1 - 1 = 0$, $f_{\text{att}} = f_{\text{def}} = 1$, and by (★) the blow of C_i delivers $\max(1, \lfloor 1 \cdot a_i \rfloor) = a_i$ (R7). (This holds regardless of relative speeds; that the player is faster is not needed.)

Every enemy is alive when struck, and dies. The blows aimed at E_g are the three a_i , $i \in G_g$, and $a_i < T/2$, so any two of them sum to less than T : E_g survives its first two blows and the third striker finds it alive. Damage accumulates in the pool (R8), and after the third blow the accumulated total is $\sum_{i \in G_g} a_i = T$, so $\text{kills} = 1$ by Lemma E.12's computation and E_g dies.

All m enemies die and the destroyed value is $m \cdot 1 = W$. ■

Lemma E.14 (game yes $\implies$ 3-PARTITION yes). *If $G_3(a, T)$ is a yes-instance then (a, T) is a 3-PARTITION yes-instance.*

Proof. Each enemy creature has value 1 and there are m of them, so a destroyed value of at least $W = m$ forces all m to die. By Lemma E.12, $\sum_{i \in S_g} a_i \geq T$ for every g , with the S_g pairwise disjoint subsets of $[3m]$. Summing,

$$mT \leq \sum_{g=1}^m \sum_{i \in S_g} a_i \leq \sum_{i=1}^{3m} a_i = mT,$$

so every inequality is tight: $\sum_{i \in S_g} a_i = T$ for each g , and the S_g cover $[3m]$. Finally $T/4 < a_i < T/2$ forces $|S_g| = 3$ —two elements sum to less than T , four to more—so $\{S_1, \dots, S_m\}$ is a 3-partition. (The S_g are allowed to be proper subsets of a group's three types, since slots may be left empty; the counting is what rules that out.) ■

Proof of Theorem 2. Lemmas E.13 and E.14 give the equivalence, and the totality branch extends it to every encoding. The construction is computable in time polynomial in the unary encoding of (a, T) and every number in the output is bounded by a polynomial in m and $\max_i a_i$, so it is a pseudo-polynomial—indeed polynomial under unary encoding—reduction; since 3-PARTITION is strongly NP-complete [GJ79], ARMY-ALLOCATION is strongly NP-hard on this family. The instances have $R = 1$, no obstacles, no abilities, one creature per enemy stack, and every player type of stock one. ■

Remark. The three rows are not decoration. In one row a hex has two neighbours, so a third stack could not reach E_g without walking through a hex occupied by an ally, and occupied hexes are not enterable (R4). The bug was found by the geometry self-check that verifies Lemma E.11 on the *built* instance with all slots occupied (Section 4).

E.5 Theorem 4 and its corollaries

Theorem 4. ARMY-ALLOCATION is **strongly** NP-hard already for instances with $R = 1$; **no obstacles and no abilities of any kind**; every player creature type of stock one, one creature per enemy stack; a rectangular open battlefield of six rows and $4m+2$ columns; and **complete reachability**—in the starting position, with every slot occupied, every player stack can attack every enemy stack.

Corollary 4.1. *The same instances are hard with the allocation given. BATTLE-PLAY is strongly NP-hard on obstacle-free boards with complete reachability.*

Corollary 4.2 (hit-point objective). *On the same instances, replace the objective by **total enemy hit points removed**—the damage the enemy stacks absorb, with overkill discarded—and the target by $W_{\text{hp}} = mT$. The problem remains strongly NP-hard, with the allocation free or given.*

The instance $G_F(a, T)$. From a 3-PARTITION instance $(a_1, \dots, a_{3m}; T)$ with $m \geq 1$:

- **Battlefield.** $h = 6$ rows ($y \in \{0, \dots, 5\}$) and $w = 4m + 2$ columns; **no obstacles**.
- **Defence.** For $g = 1, \dots, m$ put $X_g := 4g - 2$ and place E_g , **one** creature, at $e_g := (X_g, 3)$, of a type with $\text{att} = \text{def} = 1$, flat damage 1, $\text{hp} = T$, $\text{spd} = 1$, $\text{flags} = \emptyset$, $\text{value} = 1$, playing $(\dagger)$.
- **Player army.** $3m$ creature types $C_1, \dots, C_{3m}$ with $\text{att}(C_i) = \text{def}(C_i) = 1$, flat damage $\text{dmg}_{\min} = \text{dmg}_{\max} = a_i$, $\text{hp} = 5$, $\text{spd} = s := w + h = 4m + 8$, $\text{flags} = \emptyset$, $\text{value} = 0$, and stock exactly one. There are $k = 3m$ slots along the top row, $p_j := (j - 1, 0)$ for $j = 1, \dots, 3m$.
- **Question.** $R = 1$, $W = m$.

The board fits: $X_1 = 2$ and $X_m = 4m - 2 \leq w - 3$, so every e_g has all six neighbours on the board; and $3m - 1 \leq w - 1$, so the deployment row is long enough. The board has $6(4m + 2) = O(m)$ hexes and every number is polynomial in m once the a_i are, so the construction is polynomial under unary encoding. The totality branch is that of Section E.4 verbatim, with $G_F((1, 1, 1), 3)$ as the fixed yes-instance.

Three named hexes per enemy carry all the geometry:

$$q_g^1 := (X_g - 1, 2), \quad q_g^2 := (X_g, 2), \quad q_g^3 := (X_g + 1, 3). \quad (\text{Q})$$

Lemma E.15 (the three approach hexes). *Row 3 is odd, so by R1 the six neighbours of $e_g = (X_g, 3)$ are*

$$(X_g - 1, 3), (X_g + 1, 3), (X_g - 1, 2), (X_g, 2), (X_g - 1, 4), (X_g, 4). \quad (\text{N})$$

The three hexes of (Q) are among them, and the $3m$ hexes $\{q_g^r : g \in [m], r \in \{1, 2, 3\}\}$ are pairwise distinct and disjoint from the deployment row.

Proof. With $\epsilon = 1$ for the odd row 3, R1 gives upper neighbours at columns $x - 1$ and x of row 2 and lower neighbours at columns $x - 1$ and x of row 4, plus the two same-row hexes; that is (N), and q_g^1, q_g^2, q_g^3 appear in it. Within a group the three hexes differ in column or row. Across groups $|X_g - X_{g'}| \geq 4$, while the columns used by group g lie in $\{X_g - 1, X_g, X_g + 1\}$, so no two groups share a hex. All q_g^r lie in rows 2 and 3, the p_j in row 0. ■

We used the offset convention of R1 in the form stated in Section E.1: from an *odd* row the two upper neighbours sit at columns $x - 1$ and x ; from an *even* row at columns x and $x + 1$. Every neighbour computation below is in that convention.

Call a play of round 1 **attack-only** if every player stack, on its turn, either passes or issues WALK_AND_ATTACK—no MOVE-only action, no WAIT, no DEFEND. Lemmas E.16–E.19 are used *only* in the yes-direction, where the play is ours to choose, so restricting them to attack-only plays costs nothing; the no-direction (Lemma E.22) is geometry-free and quantifies over every play of the full model.

Lemma E.16 (row 1 stays clear). *In any attack-only play of round 1 of G_F , every occupied hex lies in row 0, or is an enemy hex, or is adjacent to an enemy hex. In particular **rows 1 and 5 are free throughout**.*

Proof. Enemy stacks sit on e_g in row 3 and never move: under ($\ddagger$) they issue WAIT and then DEFEND in place, and neither changes a hex (Section 2.4). A player stack starts on p_j in row 0, and in an attack-only play the only action that changes its hex is WALK_AND_ATTACK, whose destination is adjacent to the struck enemy (R11), hence adjacent to some e_g , hence by (N) in rows 2, 3 or 4. Each stack takes one terminal action (R9), so no other hex is ever entered. Rows 1 and 5 contain no e_g and, by (N), no neighbour of any e_g . ■

Lemma E.17 (routing). *Fix g and $r \in \{1, 2, 3\}$. In any position of an attack-only play of round 1 in which E_g is alive and q_g^r is free, a player stack still standing on its deployment hex p_j can move to q_g^r and strike E_g . The walk has length at most $w + 2 = 4m + 4 < s$, and this holds independently of the order in which the stacks are activated.*

Proof. By Lemma E.16 every hex of row 1 is free. Row 0 is even, so by R1 the two lower neighbours of $p_j = (j - 1, 0)$ are $(j - 1, 1)$ and $(j, 1)$: the stack leaves row 0 in one step and may then walk along row 1 freely, at most $w - 1$ steps. Row 1 is odd, so the two lower neighbours of $(x, 1)$ are $(x - 1, 2)$ and $(x, 2)$; row 2 is even, so the two lower neighbours of $(x, 2)$ are $(x, 3)$ and $(x + 1, 3)$. Hence

- $q_g^1 = (X_g - 1, 2)$ is entered from $(X_g - 1, 1)$, one step down from row 1;
- $q_g^2 = (X_g, 2)$ is entered from $(X_g, 1)$, likewise;
- $q_g^3 = (X_g + 1, 3)$ is entered from $(X_g + 1, 2)$, which is entered from $(X_g + 1, 1)$. The intermediate hex $(X_g + 1, 2)$ is *always* free: its neighbours are, by R1 for the even row 2, $(X_g, 2)$, $(X_g + 2, 2)$, $(X_g + 1, 1)$, $(X_g + 2, 1)$, $(X_g + 1, 3)$ and $(X_g + 2, 3)$, and no enemy hex $(X_{g'}, 3)$ is among them, since $X_{g'} - X_g \in 4\mathbb{Z}$ excludes $X_{g'} \in \{X_g + 1, X_g + 2\}$; so by Lemma E.16 it is never occupied.

Each route is one step out of row 0, at most $w - 1$ steps along row 1, and at most two steps down, so at most $w + 2 = 4m + 4$ steps, all over free hexes, well within $s = 4m + 8$ (R10). By R11 the stack may therefore move to q_g^r and strike the adjacent E_g (Lemma E.15). Nothing in the route depends on which other stacks have already acted, beyond the freeness of q_g^r , which is the hypothesis. ■

Lemma E.18 (complete reachability). *In the starting position of G_F , with all $3m$ slots occupied, every player stack can attack every enemy stack.*

Proof. At the start the occupied hexes are exactly row 0 and the e_g , so every q_g^1 is free; apply Lemma E.17. ■

Lemma E.19 (simultaneous realizability). *Let $\varphi : [3m] \rightarrow [m]$ satisfy $|\varphi^{-1}(g)| = 3$ for every g . Then there is an attack-only play of round 1 of G_F , with all $3m$ slots occupied, in which every stack j strikes $E_{\varphi(j)}$.*

Proof. For each g let $j_g^1 < j_g^2 < j_g^3$ be the three slots with $\varphi(j) = g$, and instruct stack j_g^r to move to q_g^r and strike E_g . This assigns the $3m$ stacks to the $3m$ pairwise distinct hexes q_g^r bijectively (Lemma E.15), so no two stacks are sent to the same hex, and by construction the only stack that ever enters q_g^r is j_g^r . Hence q_g^r is free when j_g^r acts, and Lemma E.17 makes the move and the strike legal; both are independent of the activation order, so the play is well defined whatever order R9 imposes.

Two further points make the play realize φ rather than merely attempt it. Every stack gets its action: under $(\ddagger)$ no enemy initiates, and by Lemma E.2 a stack takes at most one retaliation, of 1 point against $hp = 5$, strictly after its own blow, so no stack is killed before acting (R9). And every E_g is alive when each of its three strikers acts: the blows aimed at it are the three a_i with $\varphi(i) = g$, and $a_i < T/2$, so any two of them sum to less than $T = hp(E_g)$ and E_g survives the first two. Note that this argument uses only the 3-PARTITION promise $a_i < T/2$, so the lemma holds for *every* three-per-enemy φ , not only for those a 3-partition produces. ■

Lemma E.20 (damage accounting). *Fix any allocation and any play of round 1 of G_F —not necessarily attack-only. For $g \in [m]$ let $S_g \subseteq [3m]$ be the set of types whose stack struck E_g . Then the S_g are pairwise disjoint, the nominal damage delivered to E_g is at most $\sum_{i \in S_g} a_i$, the damage E_g absorbs is at most $\min(T, \sum_{i \in S_g} a_i)$ —with equality when no striker of E_g waited; a waiting blow meets the postponed **DEFEND** bonus and delivers at most nominal, strictly less unless the clamp at 1 binds, as it does for $a_i = 1$ —and E_g is dead at the end of the round only if $\sum_{i \in S_g} a_i \geq T$. If moreover no striker of E_g waited and $\sum_{i \in S_g} a_i \geq T$, then E_g is dead.*

Proof. Each type has stock one and a slot is homogeneous, so each type occupies at most one slot and each stack holds at most one creature. By Lemma E.1 each stack delivers at most one blow in the round—one terminal action (R9, R11), and no retaliation blow, because under $(\ddagger)$ no enemy initiates—so each stack strikes at most one enemy and the S_g are pairwise disjoint. By Lemma E.2 a player stack never loses a creature, so the stack of C_i has count = 1 when it strikes and its nominal blow is a_i under $(\star)$; by the one-round lemma of Section 2.4 it delivers at most that, and exactly that if it did not wait. Damage accumulates in E_g ’s pool and the excess is discarded (R8), and E_g is one creature of T hit points, so by R5, R6 and R8 the absorbed amount is $\min(T, \text{delivered})$ and E_g dies exactly when the absorbed amount reaches T . Both implications follow. ■

Lemma E.20 is where the “each type has stock one, so each stack strikes at most one enemy” is discharged properly. Stock one bounds the number of *slots* a type occupies; what bounds the blows is one terminal action per round together with the absence of enemy initiative. As printed there, that justification would survive an attacking defence, which the theorem does not.

Lemma E.21 (3-PARTITION yes $\implies$ game yes). *If (a, T) is a 3-PARTITION yes-instance then $G_F(a, T)$ is a yes-instance, and the witness uses the allocation $C_i \mapsto \text{slot } i$.*

Proof. Let $\{G_1, \dots, G_m\}$ be triples with $\sum_{i \in G_g} a_i = T$. Deploy C_i in slot i (any injection would do; each type has stock one, so this is feasible) and set $\varphi(i) := g$ for $i \in G_g$. By Lemma E.19 there is an attack-only play in which every stack strikes its assigned enemy; no stack waits, so by Lemma E.20 each E_g is delivered exactly $\sum_{i \in G_g} a_i = T$ and dies. The destroyed value is $m \cdot 1 = W$. ■

Lemma E.22 (game yes $\implies$ 3-PARTITION yes). *If $G_F(a, T)$ is a yes-instance then (a, T) is a 3-PARTITION yes-instance.*

Proof. Enemy creatures have value 1 and there are m of them, so a destroyed value of at least $W = m$ forces all m to die. By Lemma E.20, $\sum_{i \in S_g} a_i \geq T$ for every g , with the S_g pairwise disjoint subsets of $[3m]$; summing,

$$mT \leq \sum_{g=1}^m \sum_{i \in S_g} a_i \leq \sum_{i=1}^{3m} a_i = mT,$$

so every inequality is tight, the S_g cover $[3m]$ and each sums to T ; $T/4 < a_i < T/2$ then forces $|S_g| = 3$. This direction uses no geometry: the board can only ever *restrict* which S_g are achievable, so no freedom of movement can help. ■

Proof of Theorem 4. Lemmas E.21 and E.22 give the equivalence and the totality branch extends it to every encoding; the construction is polynomial under unary encoding, and 3-PARTITION is strongly NP-complete [GJ79], so ARMY-ALLOCATION is strongly NP-hard on this family. The instances have $R = 1$, no obstacles, no abilities, every player type of stock one, one creature per enemy stack, a $6 \times (4m + 2)$ open rectangle, and complete reachability by Lemma E.18. Membership in NP is Lemma 2.1, so the problem is strongly NP-complete on this family. ■

Proof of Corollary 4.1. Fix the allocation $C_i \mapsto \text{slot } i$. Lemma E.21 uses exactly that allocation and Lemma E.22 never mentions the allocation, so the equivalence survives with the allocation given as part of the input. ■

Proof of Corollary 4.2. The objective is now the total number of **enemy** hit points removed—the sum over enemy stacks of the damage they absorb, with overkill discarded (R8). (Player hit points are removed too, by retaliation; they are not counted, exactly as the Problem of Section 2.3 counts enemy values only.)

Fix any allocation and any play; let A_g be the damage E_g absorbs and D_g the nominal damage delivered to it—a waiting striker can deliver less than nominal, which is why the two are named apart. By Lemma E.20 the S_g are pairwise disjoint and $A_g \leq \min(T, D_g)$ with $D_g \leq \sum_{i \in S_g} a_i$. Suppose the total absorbed reaches $W_{\text{hp}} = mT$. Each of the m terms A_g is at most T , so all m are exactly T ; hence $\sum_{i \in S_g} a_i \geq D_g \geq A_g = T$, and summing over the disjoint S_g against $\sum_i a_i = mT$ makes every inequality tight, so $\sum_{i \in S_g} a_i = T$ for every g and the S_g cover $[3m]$; $T/4 < a_i < T/2$ forces $|S_g| = 3$ and $\{S_1, \dots, S_m\}$ is a 3-partition.

Conversely, the witness play of Lemma E.21 delivers exactly T to each enemy, which each absorbs in full, for a total of mT . The argument never mentions the allocation, so it holds with the allocation free or given. ■

Where the hardness lives. In G_F every type has stock one and, by Lemma E.18, every slot reaches every enemy, so every injection of types into slots is equivalent: the multiset of blows available does not depend on which slot holds which type, and by Lemma E.19 every three-per-enemy targeting is realizable from any of them. The decision that encodes the 3-partition is the choice of targets—which is Corollary 4.1, and which is what makes Theorem 4 the targeting-driven endpoint of the two-source claim.

Remark. The board does impose one thing: an enemy has six neighbours, so at most six stacks can strike it in one round. Under $(\star)$ as completed in Section E.1 this bound is correct—no striker is killed by the retaliation it draws (Lemma E.2), so no seat is ever vacated mid-round. It never binds here, three stacks per enemy, but it is why “featureless” means *complete reachability plus local seat capacity* and not “positions do not exist”.